



What is CPaskal?

CPaskal is a Pascal-flavored systems programming language with perfect C ABI interoperability. You write `.cpas` source files in a clean, Pascal-inspired syntax, and CPaskal compiles them to C++23 -- which is then compiled to native binaries via Zig/Clang. The result is a language that feels like modern Pascal but links directly against any C library without wrappers, bindings generators, or FFI overhead.

```
module exe hello;
begin
  println("Hello, CPaskal!");
end.
```

```
> cpas hello -r
Hello, CPaskal!
```

Tip

 **Fast path:** read [Getting Started](#), skim [Language Reference](#), then jump to [C/C++ Interop](#) when you are ready to call external libraries.

Documentation Roadmap

Reader Goal	Start Here	Why
 Run your first program	Getting Started	Install, first <code>.cpas</code> file, build and run
 Learn the language	Language Reference	Types, operators, routines, control flow, expressions
 Understand modules	Module System	exe, dll, lib, unit -- imports, visibility, init/final
 Call C/C++ code	C/C++ Interop	External clause, name aliasing, inline C++, CImporter
 Memory and data structures	Memory and Data Structures	Pointers, arrays, records, choices, sets, heap allocation
 Verify exact syntax	Formal Grammar	BNF rules derived from the parser
 Runtime library	Runtime Library	rt_* functions, intrinsics, exception handling
 Debug your code	Debugging	DAP protocol, breakpoints, source-level debugging

Reader Goal	Start Here	Why
 Follow conventions	Code Style	Naming, formatting, file organization
 Common tasks	Common Tasks	Practical recipes for everyday CPaskal work

Core Idea

CPaskal compiles `.cpas` source through a full compiler pipeline to native executables, shared libraries, or static libraries:

```
.cpas source -> Lexer -> Parser -> AST -> Semantics -> Codegen -> C++23 -> Zig/Clang -> native binary
```

text

The compiler is a traditional multi-pass design: the lexer tokenizes, the parser builds an AST via recursive descent with a Pratt expression parser, semantic analysis enriches the AST with resolved types and symbols, and code generation walks the enriched AST to emit C++23. Zig/Clang handles the final compilation to machine code.

Important

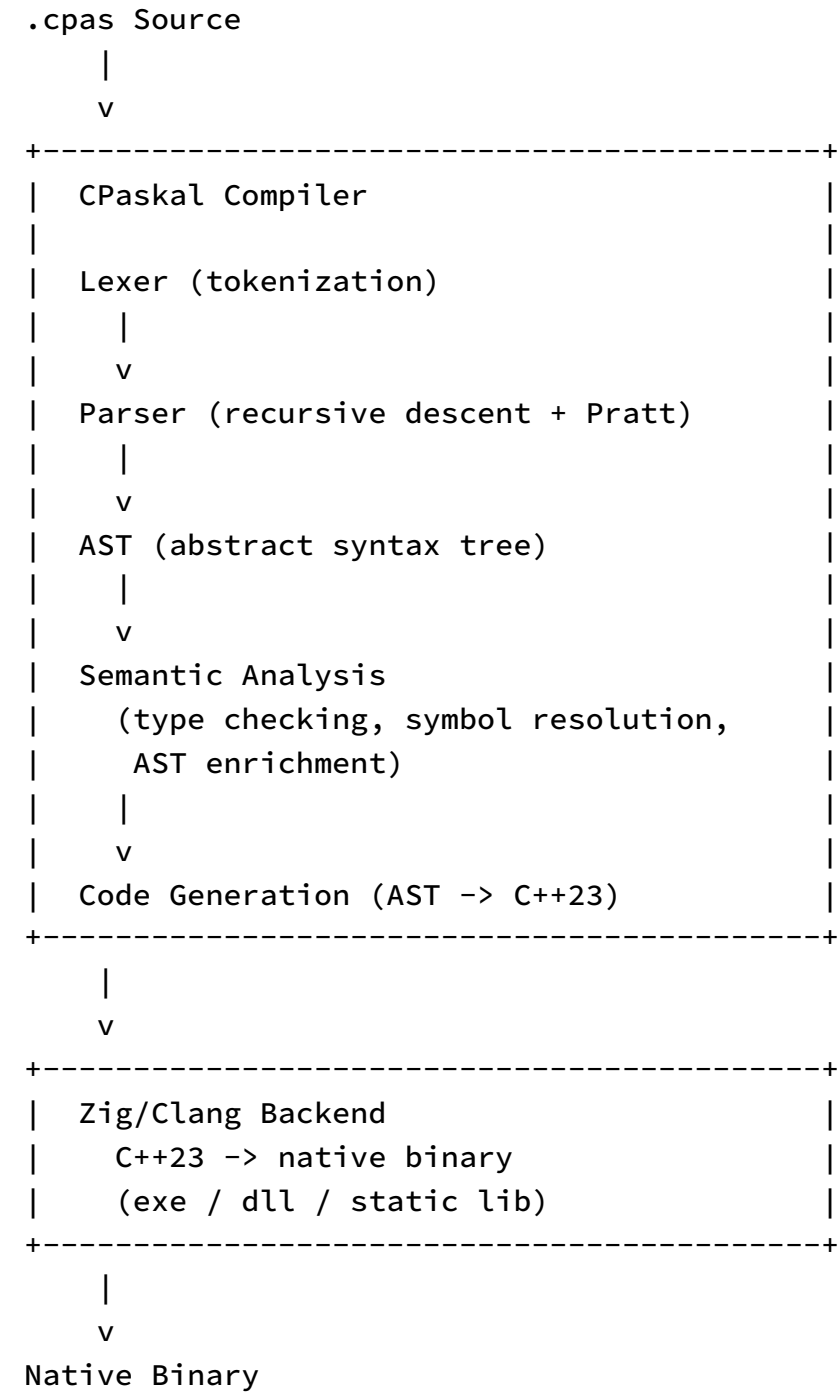
 CPaskal is a compiled language, not an interpreter or VM. Your `.cpas` source becomes a native binary -- an `.exe`, `.dll`, or `.a` file -- with no runtime interpreter or virtual machine involved.

Key Features

Feature	What It Means
 Zero-cost C ABI	Call any C library directly with <code>external "c"</code> . No wrappers, no marshaling, no overhead.
 Module system	Four module kinds: <code>exe</code> , <code>dll</code> , <code>lib</code> , <code>unit</code> . Public/private visibility, init/final blocks, topological dependency ordering.
 Rich type system	16 primitive types, records (plain, packed, aligned, derived, overlay, tagged, bitfield), choices (discriminated unions), sets, fixed and dynamic arrays, typed pointers.
 C++23 backend	Generates clean, readable C23 code. <u>Inspect the output, debug at the C level</u> if needed, or integrate with existing C++ projects.
 Zig/Clang toolchain	Cross-compilation built in. Target Windows or Linux from either platform with <code>-t x86_64_linux</code> or <code>-t x86_64_windows</code> .
 Source-level debugging	DAP protocol support with <code>@breakpoint</code> directives. Debug <code>.cpas</code> source directly in VS Code.
 Built-in testing	<code>test "name" ... end;</code> blocks with assertion intrinsics. Run with <code>@unittestmode on;</code> .
 Conditional compilation	<code>@ifdef</code> , <code>@ifndef</code> , <code>@define</code> , <code>@undef</code> , <code>@else</code> , <code>@elseif</code> , <code>@endif</code> -- platform-specific code without preprocessor complexity.
 Inline C++	<code>cpp("...")</code> expressions and <code>cppstart</code> / <code>cppend</code> blocks for when you need direct C++ access.

 Architecture

text



 Who Is This For?

- **Systems programmers** who want a Pascal-flavored alternative to C with modern features like modules, generics-ready type system, and first-class C interop.
- **Delphi/Pascal developers** who want a systems language that feels familiar but compiles to native code through a modern toolchain.
- **C library consumers** who want to call SDL3, raylib, OpenGL, or any C library without writing binding generators or FFI boilerplate.
- **Cross-platform developers** who want to target Windows and Linux from a single codebase with a single compiler invocation.

 CLI Reference

CPaskal includes a command-line compiler (`cpas`) for building and running `.cpas` projects.

Syntax:

text

```
cpas <source> [options]
cpas cimport <script> [options]
```

Build and Run:

Flag	Description
<code>-r, --run</code>	Run after successful build
<code>-d, --debug</code>	Build with debug info and launch debugger

Flag	Description
-t, --target <target>	Set compilation target (e.g. x86_64_linux)
-o, --output <path>	Set output directory
-sub, --subsystem <type>	Set subsystem: console (default) or gui
-opt, --optimize <level>	Set optimize level: debug (default), release_safe , release_fast , release_small
-h, --help	Show help

Examples:

```
cpas hello -r
cpas hello -r -t x86_64_linux
cpas hello -r -opt release_fast
cpas hello -d
```

text

 Note

 -r and -d are mutually exclusive. The debugger requires x86_64_windows target. Source filename should be without the .cpas extension. Zig/Clang backend (~350MB) is downloaded automatically on first build.

 Current Status

The compiler is working end-to-end with support for:

- 16 primitive types with full C++23 mappings
- Variables, typed/untyped constants, constant expressions
- Arithmetic, comparison, logical, bitwise, and compound assignment operators
- Control flow: if / else , while , for , repeat / until , match , guard , break , continue
- Routines: procedures, functions, overloading, const/var/out parameters, forward declarations
- Records: plain, packed, aligned, derived, overlay, tagged, bitfield, with methods and operators
- Choices (discriminated unions), sets, fixed arrays, dynamic arrays
- Typed and untyped pointers, new / dispose , getmem / freemem / resizemem
- Module system: exe, dll, lib, unit with public/private visibility
- C/C++ interop: external "c" , external "cpp" , name alias, clink / cpplink , inline C++
- Conditional compilation: #ifdef / #ifndef / #define / #undef / #else / #elseif / #endif
- Source-level debugging: DAP protocol, @breakpoint , PDB/DWARF debug info
- Built-in testing: test blocks with assertion intrinsics
- Clmporter: generate .cpas bindings from C headers
- Cross-compilation: Windows and Linux targets from either platform

 System Requirements

Area	Requirement
Operating system	Windows 10/11 x64, Linux x64
Runtime dependencies	None (native binaries)
External toolchain	Zig/Clang (~350MB, downloaded automatically)
Building the compiler from source	Delphi 12.x or higher

Table of Contents

-  **Getting Started**: installation, first program, build and run
-  **Language Reference**: types, operators, routines, control flow, expressions
-  **Module System**: exe, dll, lib, unit, imports, visibility
-  **C/C++ Interop**: external clause, inline C++, CImporter
-  **Memory and Data Structures**: pointers, arrays, records, choices, sets
-  **Formal Grammar**: BNF rules derived from the parser
-  **Runtime Library**: `rt_*` functions, intrinsics
-  **Debugging**: DAP, breakpoints, source-level debugging
-  **Code Style**: naming conventions and formatting
-  **Common Tasks**: practical recipes for everyday work

Getting Started

This section walks you through installing CPaskal, writing your first program, and building it. By the end you will have a working `.cpas` project that compiles to a native binary.

 **Tip**

 If you already know what CPaskal is, skip straight to [Your First Program](#). For a full language overview, see [What is CPaskal?](#).

Prerequisites

CPaskal compiles your `.cpas` source to C++23 and then to a native binary using Zig/Clang as the backend toolchain. You need:

Requirement	Details
Operating system	Windows 10/11 x64 or Linux x64
CPaskal compiler	The <code>cpas</code> executable (see Installation)
Zig/Clang toolchain	~350MB, downloaded automatically on first build
Disk space	~500MB total (compiler + toolchain + build artifacts)

No other runtime, SDK, or framework is required. CPaskal produces standalone native binaries with no runtime dependencies.

Installation

1. Download the latest CPaskal release from [GitHub](#).
2. Extract the archive to a directory of your choice.
3. Add the directory containing `cpas.exe` (or `cpas` on Linux) to your system `PATH`, or run it directly from its location.

That is all. The first time you build a project, the compiler will automatically download and cache the Zig/Clang toolchain (~350MB). Subsequent builds use the cached toolchain instantly.

 **Note**

 If you want to build the CPaskal compiler itself from source, you need Delphi 12.x or higher. Most users only need the pre-built `cpas` binary.

Your First Program

Create a file called `hello.cpas`:

```
module exe hello;
begin
  println("Hello, CPaskal!");
end.
```

Every CPaskal program starts with a **module declaration**: `module <kind> <name>;`. Here, `exe` means this module produces an executable, and `hello` is the module name -- which must match the filename (without extension).

The `begin...end.` block is the program's entry point, equivalent to `main()` in C. Note the period after the final `end` -- it marks the end of the module.

Building and Running

Open a terminal in the directory containing `hello.cpas` and run:

```
cpas hello -r
```

The `-r` flag tells the compiler to run the program immediately after a successful build. You should see:

```
Hello, CPaskal!
```

To build without running:

```
cpas hello
```

This produces a native executable (`hello.exe` on Windows, `hello` on Linux) in the output directory.

 **Important**

 Pass the source filename **without** the `.cpas` extension. The compiler adds it automatically.

Cross-Compilation

CPaskal can target Windows or Linux from either platform. Use the `-t` flag to set the target:

```
cpas hello -r -t x86_64_windows
cpas hello -r -t x86_64_linux
```

The default target matches your current platform.

Optimization Levels

By default, CPaskal builds in debug mode. Use `-opt` to set the optimization level:

Level	Flag	Description
Debug	<code>-opt debug</code>	No optimization, full debug info (default)
Release Safe	<code>-opt release_safe</code>	Optimized with safety checks
Release Fast	<code>-opt release_fast</code>	Maximum optimization
Release Small	<code>-opt release_small</code>	Optimized for binary size

```
cpas hello -r -opt release_fast
```

Debugging

To build with debug info and launch the debugger:


```
cpas hello -d
```

text

This starts a DAP-compatible debug session. You can set breakpoints in your `.cpas` source using the `@breakpoint` directive:

```
module exe debugdemo;
begin
  var x: int32 = 42;
  @breakpoint;
  println("x = {}", x);
end.
```

 **Note**

 The debugger currently requires the `x86_64_windows` target. The `-d` and `-r` flags are mutually exclusive.

Module Kinds

CPaskal has four module kinds, each producing a different output:

Kind	Declaration	Output	Entry Point
exe	<code>module exe myapp;</code>	Executable (.exe/.elf)	<code>begin...end.</code>
dll	<code>module dll mylib;</code>	Shared library (.dll/.so)	<code>end.</code> (no main body)
lib	<code>module lib mylib;</code>	Static library (.a/.lib)	<code>end.</code> (no main body)
unit	<code>module unit myutil;</code>	Compiled inline	<code>end.</code> (no main body)

Only `exe` modules have a `begin...end.` main body. Library and unit modules end with just `end.` and expose functionality through `public` routines, types, and constants.

Importing Modules

To use code from another module, import it:

```
module exe myapp;
import mathlib;
begin
  println!("{}", mathlib.add(2, 3));
end.
```

All public symbols from imported modules must be **fully qualified** with the module name: `mathlib.add`, `mathlib.MY_CONST`, `mathlib.MyType`. Unqualified access is a compile error.

Project Structure

A typical CPaskal project looks like this:

```
myproject/
  myapp.cpas      -- main executable module
  mathlib.cpas    -- utility unit module
  graphics.cpas   -- another unit module
```

text

Each `.cpas` file is one module. The module name in the declaration must match the filename. The compiler resolves imports automatically and compiles dependencies in the correct order.

Initialize and Finalize

Any module can have `initialize` and `finalize` blocks that run at startup and shutdown:

```

module unit myutil;

public routine helper(): int32;
begin
  return 42;
end;

initialize
  println("myutil loaded");
end;

finalize
  println("myutil unloaded");
end;

end.

```

`initialize` runs before the main `begin` block. `finalize` runs after the main program finishes. Both are optional.

Built-in Testing

CPaskal has built-in test support. Add `@unittestmode on;` and place test blocks after `end.:`

```

module exe mylib;
@unittestmode on;

routine double(const n: int32): int32;
begin
  return n * 2;
end;

end.

test "double returns correct values"
begin
  asserteq(4, double(2));
  asserteq(0, double(0));
  asserteq(-6, double(-3));
end;

test "double handles large values"
begin
  asserteq(2000000, double(1000000));
end;

```

When `@unittestmode` is on, the test runner replaces the normal entry point. Tests run with assertion intrinsics like `asserteq`, `asserttrue`, `assertfalse`, `assertnil`, and `assertfail`.

Error Messages

CPaskal provides clear error messages with source locations:

```

hello.cpas(5,10): error CP_SEM_001: undeclared identifier 'x'
hello.cpas(8,3): error CP_SEM_042: type mismatch: expected 'int32', got 'string'

```

text

Each message includes the file, line, column, error code, and a description of the problem.

Quick Reference Card

Module declaration:


```

module exe myapp;      // executable
module dll mylib;      // shared library
module unit myutil;    // inline unit
module lib mystaticlib; // static library

```

Common CLI commands:

```

cpas myapp -r          -- build and run
cpas myapp -d          -- build and debug
cpas myapp -r -t x86_64_linux -- cross-compile to Linux
cpas myapp -r -opt release_fast -- optimized build
cpas myapp -h          -- show help

```

text

Essential syntax:

```

// Variables and constants
var x: int32 = 10;
const MAX: int32 = 100;

// Output (C++23 std::format syntax)
println("x = {}, max = {}", x, MAX);

// Control flow (always terminated with end;)
if x > 0 then
  println("positive");
end;

while x > 0 do
  x -= 1;
end;

for i := 1 to 10 do
  println!("{}", i);
end;

// Routines
routine add(a: int32; b: int32): int32;
begin
  return a + b;
end;

```

💡 Tip

💡 Continue to the [Language Reference](#) for the full syntax, or jump to [C/C++ Interop](#) to start calling C libraries.

📖 Language Reference

This section covers every language construct in CPaskal: types, literals, variables, constants, operators, control flow, routines, and expressions. For module-level features (imports, visibility, init/final), see [Module System](#). For pointers, arrays, records, and heap allocation, see [Memory and Data Structures](#).

💡 Tip

💡 CPaskal keywords are case-insensitive (canonically lowercase). Identifiers are case-sensitive. There is no **T** prefix convention on type names -- use PascalCase for types, camelCase for variables, and UPPER_CASE for constants.

12 34 Primitive Types

CPaskal has 16 built-in primitive types. All are reserved words and cannot be used as identifiers.

Integer Types

Type	Size	Range	C++23 Mapping
int8	1 byte	-128 to 127	int8_t
int16	2 bytes	-32,768 to 32,767	int16_t
int32	4 bytes	-2 ³¹ to 2 ³¹ -1	int32_t
int64	8 bytes	-2 ⁶³ to 2 ⁶³ -1	int64_t
uint8	1 byte	0 to 255	uint8_t
uint16	2 bytes	0 to 65,535	uint16_t
uint32	4 bytes	0 to 2 ³² -1	uint32_t
uint64	8 bytes	0 to 2 ⁶⁴ -1	uint64_t

Floating-Point Types

Type	Size	Description	C++23 Mapping
float32	4 bytes	32-bit IEEE 754	float
float64	8 bytes	64-bit IEEE 754	double

Other Types

Type	Size	Description	C++23 Mapping
boolean	1 byte	true or false	bool
char	1 byte	8-bit character (UTF-8)	char
wchar	2 bytes	16-bit wide character	char16_t
string	varies	Managed UTF-8 string	std::string
wstring	varies	Managed UTF-16 string	std::wstring
pointer	8 bytes	Untyped pointer	void*

 Literals

Integer Literals

```
var x: int32 = 42;           // decimal
var h: int32 = 0xFF;         // hexadecimal (0x prefix)
```

Untyped integer literals default to `int32`.

Floating-Point Literals

```
var a: float64 = 3.14;       // contextual (float32 or float64 depending on target type)
var b: float32 = 3.14f;      // explicit float32 (f suffix)
```

String Literals

Strings use double quotes with C-style escape sequences:

```
var s: string = "hello world";
var escaped: string = "line1\nline2\ttab";
```

Escape	Meaning
<code>\n</code>	Newline
<code>\t</code>	Tab
<code>\r</code>	Carriage return
<code>\0</code>	Null character
<code>\\</code>	Backslash
<code>\'</code>	Single quote
<code>\"</code>	Double quote
<code>\xHH</code>	Hex byte value

Wide String Literals

Prefix a string with lowercase `w` for UTF-16:

```
var ws: wstring = w"hello wide world";
```

Wide strings support the same escape sequences as regular strings.

Boolean Literals

```
var flag: boolean = true;
var done: boolean = false;
```

Nil Literal

```
var p: pointer = nil;           // null pointer
```

Character Assignment

Characters are assigned from single-character strings. The compiler checks that the string contains exactly one character:

```
var c: char = "A";              // single UTF-8 character
var wc: wchar = w"X";           // single UTF-16 character
```

Variables

Variables are declared with `var` and an optional initializer:

```
var x: int32 = 10;               // with initializer
var y: int32;                    // zero-initialized
var name: string = "Alice";
```

Variables can also be declared inline within statement blocks:

```
begin
  var sum: int32 = a + b;
  println("sum = {}", sum);
end.
```

Constants

Constants are declared with `const` . They can be typed or untyped:

```
const
  MAX: int32 = 100;           // typed constant
  PI: float64 = 3.14159;
  GREETING = "hello";        // untyped (type inferred)
```

Constant expressions are evaluated at compile time:

```
const
  DOUBLED = 21 * 2;           // expression constant (42)
  IS_EQ = 5 = 5;              // boolean expression constant (true)
```

+ Operators

Arithmetic Operators

Operator	Meaning	Example
<code>+</code>	Addition	<code>a + b</code>
<code>-</code>	Subtraction / unary negation	<code>a - b</code> , <code>-x</code>
<code>*</code>	Multiplication	<code>a * b</code>
<code>/</code>	Division	<code>a / b</code>
<code>div</code>	Integer division	<code>a div b</code>
<code>mod</code>	Modulo	<code>a mod b</code>

Comparison Operators

Operator	Meaning	Example
<code>=</code>	Equal	<code>a = b</code>
<code><></code>	Not equal	<code>a <> b</code>
<code><</code>	Less than	<code>a < b</code>
<code>></code>	Greater than	<code>a > b</code>
<code><=</code>	Less or equal	<code>a <= b</code>
<code>>=</code>	Greater or equal	<code>a >= b</code>

Logical and Bitwise Operators

Operator	Meaning	Example
<code>and</code>	Logical/bitwise AND	<code>a and b</code>
<code>or</code>	Logical/bitwise OR	<code>a or b</code>
<code>xor</code>	Logical/bitwise XOR	<code>a xor b</code>
<code>not</code>	Logical/bitwise NOT	<code>not a</code>
<code>shl</code>	Bit shift left	<code>a shl 2</code>
<code>shr</code>	Bit shift right	<code>a shr 2</code>

Compound Assignment Operators

Operator	Equivalent	Example
<code>+=</code>	<code>x := x + y</code>	<code>x += 1</code>
<code>-=</code>	<code>x := x - y</code>	<code>x -= 1</code>
<code>*=</code>	<code>x := x * y</code>	<code>x *= 2</code>
<code>/=</code>	<code>x := x / y</code>	<code>x /= 2</code>

Other Operators

Operator	Meaning	Example
<code>:=</code>	Assignment	<code>x := 42</code>
<code>^</code>	Pointer dereference (postfix)	<code>p^</code>
<code>address of</code>	Address-of (prefix)	<code>address of x</code>
<code>in</code>	Set membership	<code>5 in mySet</code>

Operator Precedence

From highest to lowest:

Level	Operators
1 (highest)	<code>not</code> , unary <code>-</code> , unary <code>+</code> , <code>address of</code>
2	<code>*</code> , <code>/</code> , <code>div</code> , <code>mod</code> , <code>and</code> , <code>shl</code> , <code>shr</code>
3	<code>+</code> , <code>-</code> , <code>or</code> , <code>xor</code>
4 (lowest)	<code>=</code> , <code><></code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>in</code>

Use parentheses to override precedence when needed.

 Control Flow

All control-flow statements in CPaskal are terminated with `end;` (except `repeat...until`). No parentheses are required around conditions.

if / then / else / end

```
if x > 0 then
  println("positive");
end;

if x > 0 then
  println("positive");
else
  println("non-positive");
end;
```

while / do / end

```
while x > 0 do
  x -= 1;
end;
```

for / to / downto / do / end

```
for i := 1 to 10 do
  println("{} ", i);
end;

for i := 10 downto 1 do
  println("{} ", i);
end;
```

The iterator variable must be declared before the loop. `continue` performs the iterator step before re-testing the condition.

repeat / until

```
repeat
  x += 1;
until x >= 10;
```

The body executes at least once. Note: `repeat...until` does **not** use `end;` -- the `until` keyword closes the loop.

match / of / end

```
match value of
  1: println("one");
  2, 3: println("two or three");
  4..10: println("four through ten");
else
  println("something else");
end;
```

`match` supports single values, comma-separated value lists, and ranges (`low..high`). The `else` branch handles values not matched by any case.

break and continue

```
while true do
  if done then
    break;           // exit innermost loop
  end;
  if skip then
    continue;        // next iteration
  end;
  // ...
end;
```

Both are valid only inside `while` , `for` , and `repeat` loops.

Exception Handling

guard / except / finally / end

```
guard
  // protected code
except
  println("error: code={}, msg={}", exccode(), excmsg());
finally
  // always runs, even if no exception
end;
```

You can use `except` only, `finally` only, or both. The `guard` block catches both software exceptions (`throw` / `throwcode`) and hardware exceptions (division by zero, access violations, stack overflow, etc.).

throw / throwcode

```
throw("something went wrong");           // code defaults to 1 (RT_EXC_SOFTWARE)
throwcode(42, "custom error");           // user-defined error code
```

Exception Intrinsic

Intrinsic	Returns	Description
exccode()	int32	Error code of the last exception
excmsg()	string	Error message of the last exception

 Routines

Routines are declared with the `routine` keyword. A routine without a return type is a procedure; with a return type it is a function.

Procedures

```
routine greet(const name: string);
begin
  println("Hello, {}", name);
end;
```

Functions

```
routine add(a: int32; b: int32): int32;
begin
  return a + b;
end;
```

 Important

 Parameters are separated by semicolons (;), not commas. Use `return` to return a value from a function.

Parameter Modifiers

Modifier	Behavior	When to Use
(none)	Pass by value	Default -- caller's value is copied
const	Immutable by value	When the routine should not modify the parameter
var	Pass by reference	When the routine needs to modify the caller's variable

```
routine swap(var a: int32; var b: int32);
var temp: int32;
begin
  temp := a;
  a := b;
  b := temp;
end;
```

 Note

 `const` parameters are immutable copies (pass by value), the same as in Delphi. They are **not** passed by reference.

Local Declarations

Routines can contain their own `const`, `type`, and `var` sections:

```
routine compute(): int32;
const
  LOCAL_CONST: int32 = 10;
type
  LocalRec = record val: int32; end;
var
  x: int32;
begin
  x := LOCAL_CONST * 2;
  return x;
end;
```

Overloading

Routines can be overloaded by parameter types. Overloaded routines require `cpplink` linkage:

```
routine cpplink add(a: int32; b: int32): int32;
begin return a + b; end;

routine cpplink add(a: float64; b: float64): float64;
begin return a + b; end;
```

If you omit `cpplink` on an overloaded routine, the compiler will auto-promote it with a warning.

Forward Declarations

Declare a routine's signature before its implementation:

```
forward routine my_func(a: int32): int32;

// ... other code ...

routine my_func(a: int32): int32;
begin
  return a * 2;
end;
```

The full declaration must appear later in the same module.

Type Declarations

Use the `type` section to declare type aliases, records, choices, and other named types:

```
type
  Age = int32;                      // type alias
  IntPtr = pointer to int32;        // typed pointer
  ConstPtr = pointer to const int32;
  Callback = routine(x: int32): int32; // routine type (function pointer)
```

Forward Type Declarations

Forward-declared types can only be used in `pointer to` contexts until fully defined:


```
forward type MyRecord;

type
  MyRecordPtr = pointer to MyRecord;    // OK -- pointer context

// Full declaration must appear later:
type
  MyRecord = record
    val: int32;
  end;
```

Type Casts

Explicit type conversions use the target type as a function:

```
var x: int64 = 12345;
var y: int32 = int32(x);           // narrow cast
var f: float64 = 3.14;
var i: int32 = int32(f);           // float to int (truncates)
```

Type Promotion Rules

When mixing types in expressions, the compiler promotes automatically:

Expression	Result Type
int + float	float (int promoted)
smaller int + larger int	larger int
float32 + float64	float64
boolean ops (and , or , xor , not)	boolean
comparisons (= , <> , < , etc.)	boolean
string + string	same string type

Comments

```
// This is a line comment

/* This is a block comment.
   Block comments can span multiple lines.
   /* They can also be nested. */
*/
```

CPaskal supports `//` line comments and `/* */` block comments. Block comments are nestable. The Pascal-style `{ }` and `(* *)` comment forms are **not** supported.

Intrinsic Functions

These built-in functions are available without imports:

Value Intrinsics

Intrinsic	Description	Example
len(expr)	Length of string, wstring, or dynamic array	len("hello") returns 5
size(type_or_expr)	Byte size of a type or expression	size(int32) returns 4

Intrinsic	Description	Example
<code>paramcount()</code>	Number of CLI arguments (excludes program name)	<code>paramcount()</code>
<code>paramstr(index)</code>	CLI argument by index (0 = program name)	<code>paramstr(0)</code>
<code>exccode()</code>	Last exception code	<code>exccode()</code>
<code>excmsg()</code>	Last exception message	<code>excmsg()</code>

String Conversion Intrinsics

Intrinsic	Description	Example
<code>utf8(wstring)</code>	Convert wstring to owned UTF-8 buffer	<code>utf8(ws)</code>
<code>cstr(string)</code>	Borrowed raw <code>char*</code> into managed string	<code>cstr(s)</code>
<code>wstr(string)</code>	Borrowed <code>wchar*</code> with runtime caching	<code>wstr(s)</code>

Statement-Level Intrinsics

These appear as statements, not expressions:

Intrinsic	Description	Example
<code>new(ptr)</code>	Allocate and default-construct typed pointer	<code>new(p)</code>
<code>dispose(ptr)</code>	Free typed pointer and set to nil	<code>dispose(p)</code>
<code>getmem(ptr)</code>	Allocate raw memory	<code>getmem(buf)</code>
<code>freemem(ptr)</code>	Free raw memory	<code>freemem(buf)</code>
<code>resizemem(ptr, size)</code>	Resize raw memory	<code>resizemem(buf, 1024)</code>
<code>setlength(arr, count)</code>	Resize dynamic array	<code>setlength(arr, 10)</code>

Output Intrinsics

Intrinsic	Description	Example
<code>print(fmt, args...)</code>	Formatted output, no newline	<code>print("x={}", x)</code>
<code>println(fmt, args...)</code>	Formatted output with newline	<code>println("x={}", x)</code>

Output uses C++23 `std::format` syntax with `{}` placeholders.

 Conditional Compilation

CPaskal supports compile-time conditional compilation with `@` directives. These are processed at the parser level and do **not** end with semicolons:

```
@define MY_FEATURE

#ifdef MY_FEATURE
    println("feature enabled");
#endif

#ifndef SOME_FLAG
    println("flag not set");
#endif

#ifdef TARGET_WIN64
    println("Windows");
#elif TARGET_LINUX64
    println("Linux");
#else
    println("Unknown platform");
#endif
```

Directive Reference

Directive	Purpose
@define SYMBOL	Define a compilation symbol
@undef SYMBOL	Undefine a compilation symbol
#ifdef SYMBOL	Compile if symbol is defined
#ifndef SYMBOL	Compile if symbol is not defined
@else	Alternate branch
@elseif SYMBOL	Alternate branch with condition
@endif	End conditional block

Conditional blocks can be nested.

Predefined Symbols

Symbol	When Defined
CPASKAL	Always
CPUX64	Always
APPTYPE_CONSOLE	Always
WINDOWS , MSWINDOWS , WIN64 , TARGET_WIN64	Windows target
LINUX , TARGET_LINUX64	Linux target
DEBUG	Debug builds (optimization = none)
RELEASE	Release builds (any optimization)
BUILD_EXE	Module kind is exe
BUILD_DLL	Module kind is dll
BUILD_LIB	Module kind is lib

 Module-Level Directives

These directives configure the build at the module level and end with a semicolon:

Directive	Value	Purpose
@target	win64 or linux64	Set compilation target
@optimize	debug , none , basic , full	Optimization level
@subsystem	console or gui	Application subsystem
@outputpath	"path"	Output directory
@modulepath	"path"	Module search path
@librarypath	"path"	Library search path
@includepath	"path"	Include search path
@addlinklibrary	"path"	Link additional library
@copydll	"path"	Copy DLL/SO to output on build
@resfile	"path"	Compiled resource file to link
@exeicon	"path"	Application icon (Windows EXE only)
@unittestmode	on or off	Enable test compilation
@breakpoint	(none)	Set debugger breakpoint
@message	hint\ warn\ error\ fatal "text"	Compile-time diagnostic

Path Prefixes

Prefix	Base Directory
\$P:	Compiler executable directory
\$D:	Current working directory
\$S:	Declaring module's directory (default)

Version Info Directives

For Windows executables, you can embed version information:

```
@addverinfo on;
@vimajor 1;
@viminor 0;
@vipatch 0;
@viproductname "My Application";
@videscription "A CPaskal application";
@vifilename "myapp.exe";
@vicompanyname "My Company";
@vicopyright "Copyright 2026";
```

 Testing

CPaskal has built-in testing support. Enable it with @unittestmode on; and place test blocks after the module's end. :

```
module exe mylib;
@unittestmode on;

routine double(const n: int32): int32;
begin
  return n * 2;
end;

end.

test "double returns correct values"
begin
  asserteq(4, double(2));
  asserteq(0, double(0));
  asserteq(-6, double(-3));
end;
```

When `@unittestmode` is on, the test runner replaces the normal program entry point.

Assertion Functions

Assertion	Purpose	Example
<code>assert(expr)</code>	Fail if false	<code>assert(x > 0)</code>
<code>asserttrue(expr)</code>	Fail if not true	<code>asserttrue(flag)</code>
<code>assertfalse(expr)</code>	Fail if not false	<code>assertfalse(err)</code>
<code>asserteq(expected, actual)</code>	Fail if not equal	<code>asserteq(5, result)</code>
<code>asserteqf(expected, actual, epsilon)</code>	Float equality within tolerance	<code>asserteqf(3.14, pi, 0.01)</code>
<code>assertnil(expr)</code>	Fail if not nil	<code>assertnil(p)</code>
<code>assertnotnil(expr)</code>	Fail if nil	<code>assertnotnil(p)</code>
<code>assertfail("msg")</code>	Unconditional failure	<code>assertfail("not done")</code>

All assertions are non-aborting -- failures accumulate and are reported at the end of the test run.

 **Tip**

 Test blocks can include their own `var` sections for local variables. Each test runs independently.

 [Complete Example](#)

Here is a complete program demonstrating variables, constants, routines, control flow, and output:

```

module exe demo;

const
  MAX_ITEMS: int32 = 5;

routine factorial(n: int32): int32;
var
  result: int32 = 1;
  i: int32;
begin
  for i := 1 to n do
    result *= i;
  end;
  return result;
end;

routine classify(n: int32): string;
begin
  match n of
    0: return "zero";
    1..9: return "single digit";
    10..99: return "double digit";
  else
    return "large";
  end;
end;

begin
  var i: int32;
  for i := 0 to MAX_ITEMS do
    println("{}! = {}, classified as: {}",
      i, factorial(i), classify(i));
  end;
end.

```

Tip

 Continue to [Module System](#) for imports, visibility, and multi-module projects, or jump to [C/C++ Interop](#) to start calling C libraries.

Module System

CPaskal organizes code into modules. Every `.cpas` source file is a module, and the first line declares what kind of module it is. Modules can import other modules, control which symbols are visible to importers, and define startup/shutdown logic.

Tip

 All imported symbols must be fully qualified with the module name: `myunit.my_func()`, not just `my_func()`. This keeps code explicit and avoids name collisions across modules.

Module Declaration

Every `.cpas` file begins with a module declaration:

```
module <kind> <name>;
```

The kind comes first, then the name. The name must match the filename (without the `.cpas` extension). For example, a file named `mathlib.cpas` must declare `module unit mathlib;`.

Module Kinds

CPaskal has four module kinds, each producing a different output:

Kind	Output	Main Body	Use Case
<code>exe</code>	Native executable	<code>begin...end.</code>	Standalone programs
<code>dll</code>	Shared library (.dll/.so)	<code>end.</code> (no main body)	Dynamically loaded libraries
<code>lib</code>	Static library (.a/.lib)	<code>end.</code> (no main body)	Statically linked libraries
<code>unit</code>	Compiled inline	<code>end.</code> (no main body)	Reusable code imported into other modules

Note

The kind keywords (`exe` , `dll` , `lib` , `unit`) are contextual -- they are ordinary identifiers everywhere except in the module declaration position.

Executable Module

An `exe` module is a standalone program with a `begin...end.` main body:

```
module exe hello;

begin
  println("Hello, CPaskal!");
end.
```

The `begin...end.` block is the program entry point. Only `exe` modules have a main body.

Unit Module

A `unit` module contains reusable declarations (routines, types, constants, variables) that other modules can import:

```
module unit mathlib;

public routine add(a: int32; b: int32): int32;
begin
  return a + b;
end;

public routine multiply(a: int32; b: int32): int32;
begin
  return a * b;
end;

end.
```

Units are compiled inline -- their code is incorporated directly into the importing module's output. There is no separate compiled artifact.

DLL Module

A `dll` module produces a shared library (`.dll` on Windows, `.so` on Linux):

```
module dll mylib;

public routine clink my_func(const x: int32): int32;
begin
    return x * 2;
end;

initialize
    println("dll loaded");
end;

end.
```

DLL routines that should be callable from other programs or languages need the `clink` (or `cpplink`) linkage specifier and `public` visibility.

Static Library Module

A `lib` module produces a static library (`.lib` on Windows, `.a` on Linux):

```
module lib mystaticlib;

public routine clink helper(const x: int32): int32;
begin
    return x + 1;
end;

end.
```

Static libraries are linked directly into the consuming executable at build time.



 Imports

Modules import other modules with the `import` declaration:

```
import mathlib;
import utils, helpers;    // comma-separated
```

Import declarations appear after the module declaration and before any other declarations. The compiler resolves imported modules by searching for the corresponding `.cpas` file.

Module Search Paths

The compiler searches for imported modules in this order:

- 1. The directory containing the importing module
- 2. Paths added via `@modulepath "path";` directives
- 3. The compiler's built-in search paths

Use `@modulepath` to add custom search directories:

```
@modulepath "$S:../shared";    // relative to the source file
@modulepath "$P:lib";          // relative to the compiler executable
```

Path Prefix	Base Directory
<code>\$S:</code>	Declaring module's directory (default)
<code>\$P:</code>	Compiler executable directory
<code>\$D:</code>	Current working directory

Visibility

By default, all declarations in a module are private -- visible only within that module. The `public` keyword makes a declaration visible to importing modules:

```
// Visible to importers
public const MAX_SIZE: int32 = 1024;
public type Point = record x: int32; y: int32; end;
public routine calculate(a: int32): int32;
begin return a * 2; end;
public var globalCounter: int32;

// Private (default) -- not visible to importers
const INTERNAL_LIMIT: int32 = 50;
type InternalState = record val: int32; end;
routine helper(): int32;
begin return 0; end;
var scratch: int32;
```

The `public` keyword applies to constants, types, routines, and variables.

Cross-Module Symbol Access

All symbols from imported modules must be accessed with full module qualification:

```
module exe main;
import mathlib;

begin
  var result: int32 = mathlib.add(10, 20);
  println("sum = {}", result);

  var p: mathlib.Point;
  p.x := 5;
  p.y := 10;

  println("max = {}", mathlib.MAX_SIZE);
end.
```

Unqualified access to imported symbols is a compile error. This is intentional -- it prevents ambiguity when multiple imports export symbols with the same name, and it makes every reference self-documenting.

Important

There is no `use` or `from X import Y` syntax. You always write `module.symbol`. This keeps the origin of every symbol immediately visible in the source.

Initialize and Finalize

Modules can define startup and shutdown blocks that run automatically:

```

module unit resources;

var handle: int32;

public routine get_handle(): int32;
begin return handle; end;

initialize
  handle := 42;
  println("resources initialized");
end;

finalize
  handle := 0;
  println("resources cleaned up");
end;

end.

```

- `initialize` runs at program startup, before the `begin...end.` main body of the `exe` module. For imported units, initialization runs in dependency order -- if module A imports module B, B's `initialize` runs before A's.
- `finalize` runs at program shutdown, after the main body completes. Finalization runs in reverse dependency order -- if module A imports module B, A's `finalize` runs before B's.

Both blocks are optional. All four module kinds (`exe`, `dll`, `lib`, `unit`) support `initialize` and `finalize`.

Tip

 Use `initialize` for one-time setup like opening files, allocating resources, or registering callbacks. Use `finalize` for cleanup like closing handles or freeing memory.

Unit Testing

CPaskal has built-in test support. Test blocks are defined after the module's `end.` terminator and are only compiled when unit test mode is enabled:

```
module exe mylib;
@unittestmode on;

routine add(const a: int32; const b: int32): int32;
begin
    return a + b;
end;

routine multiply(const a: int32; const b: int32): int32;
begin
    return a * b;
end;

end.

test "add returns correct sum"
begin
    asserteq(5, add(2, 3));
    asserteq(0, add(-5, 5));
    asserteq(-8, add(-5, -3));
end;

test "multiply works correctly"
begin
    asserteq(6, multiply(2, 3));
    asserteq(0, multiply(0, 100));
end;
```

Enabling Tests

Add `@unittestmode on;` before any declarations. When enabled, the test runner replaces the normal entry point -- the `begin...end.` main body is not executed.

Test Block Syntax

```
test "descriptive test name"
var
    // optional local variables
begin
    // test body with assertions
end;
```

Test blocks can contain local variable, constant, and type declarations before the `begin` keyword, just like routines.

Assertion Functions

Function	Purpose
<code>assert(expr)</code>	Fail if <code>expr</code> is false
<code>asserttrue(expr)</code>	Fail if not true
<code>assertfalse(expr)</code>	Fail if not false
<code>asserteq(expected, actual)</code>	Fail if not equal (type-dispatched)
<code>asserteqf(expected, actual, epsilon)</code>	Float equality within tolerance
<code>assertnil(expr)</code>	Fail if not nil
<code>assertnotnil(expr)</code>	Fail if nil
<code>assertfail("msg")</code>	Unconditional failure

All assertions are non-aborting -- a failed assertion records the failure but continues executing the test. Failures accumulate and are reported at the end.

Dependency Ordering

The compiler automatically determines the correct order to process modules using topological sorting. If module A imports module B, the compiler ensures B is fully processed (parsed, analyzed) before A.

Circular dependencies between modules are not allowed -- they produce a compile error.

Consuming DLLs and Static Libraries

Building and Consuming a DLL

First, build the DLL module:

```
// mylib.cpas
module dll mylib;

public routine clink double_it(const x: int32): int32;
begin
  return x * 2;
end;

end.
```

Build it: `cpas mylib`

Then consume it from an executable:

```
// app.cpas
module exe app;

// Declare the external function, linking against "mylib"
routine clink double_it(const x: int32): int32; external "mylib";

begin
  println("{} ", double_it(21)); // prints 42
end.
```

The `external "mylib"` clause tells the compiler to link against the `mylib` shared library. At runtime, the DLL/SO must be available in the system's library search path or alongside the executable.

Tip

 Use `@copydll "path/to/mylib.dll";` in the consumer to automatically copy the DLL to the output directory during build.

Building and Consuming a Static Library

Build the static library:

```
// helpers.cpas
module lib helpers;

public routine clink helper_add(const a: int32; const b: int32): int32;
begin
  return a + b;
end;

end.
```

Build it: `cpas helpers`

Consume it:

```
// app.cpas
module exe app;
@librarypath "$S:../output";

routine clink helper_add(const a: int32; const b: int32): int32; external "helpers";

begin
  println("{} ", helper_add(10, 20));  // prints 30
end.
```

Use `@librarypath` to tell the compiler where to find the static library file.

❏ Module Structure Summary

Every module follows this structure, with optional sections omitted as needed:

```
module <kind> <name>;           // required: module declaration

// directives
@target x86_64_windows;         // optional: build directives
@unittestmode on;               // optional: enable tests

// imports
import unit_a;                  // optional: import other modules
import unit_b;

// declarations (in any order)
const ... ;                     // constants
type ... ;                     // type declarations
var ... ;                       // variables
routine ... ;                   // routines

// lifecycle
initialize ... end;             // optional: startup code
finalize ... end;               // optional: shutdown code

// main body (exe only) or terminator
begin                           // exe: main entry point
  ...
end.

// OR for dll/lib/unit:
end.                             // module terminator

// tests (after end.)
test "name" begin ... end;      // optional: test blocks
```

Note

The `end.` terminator (with period) marks the end of the module. Everything after it is test blocks, which are only compiled with `@unittestmode on;`.

C/C++ Interop

CPaskal provides zero-cost interoperability with C and C++ libraries. You can call any C function, link against shared or static libraries, inject raw C++ code, and even generate CPaskal bindings from C headers automatically. Because CPaskal compiles to C++23, the interop boundary is a compile-time mapping -- there is no runtime marshalling or FFI overhead.

 Tip

 The golden rule of CPaskal interop: don't use stdlib function names directly as your CPaskal routine names. They collide with runtime headers. Always use the `name` clause to alias to the actual C symbol.

 The External Clause

The `external` clause declares a routine whose implementation lives in an external C or C++ library. CPaskal does not generate a body for it -- instead, it links against the named library at compile time.

```
routine clink myabs(const n: int32): int32; external "c" name "abs";
```

This declares a CPaskal routine `myabs` that calls the C standard library function `abs`. Let's break down each part:

Part	Purpose
<code>routine</code>	Declares a callable routine
<code>clink</code>	Use C calling convention (no name mangling)
<code>myabs</code>	The CPaskal name you call in your code
<code>(const n: int32): int32</code>	Parameters and return type
<code>external "c"</code>	Link against the C standard library
<code>name "abs"</code>	The actual C symbol name to call

 Note

The `name` clause is a contextual keyword -- it has special meaning only inside the external clause and can be used as an ordinary identifier elsewhere.

Library Name

The string after `external` identifies which library to link:

Library String	Meaning
<code>"c"</code>	C standard library (libc) -- works on both Windows and Linux through Zig
<code>"kernel32"</code>	Windows system DLL
<code>"mylib"</code>	Your own library (probes for <code>.lib</code> / <code>.a</code> first, then <code>.dll</code> / <code>.so</code>)
<i>(omitted)</i>	Default to libc

You can also use a constant for the library name:

```
const EXT_LIB = "c";
routine clink mytoupper(const c: int32): int32; external EXT_LIB name "toupper";
```

Or omit the library name entirely to default to libc:

```
routine clink myabs2(const n: int32): int32; external name "abs";
```

The name Clause

The `name` clause maps your CPaskal routine name to the actual symbol in the external library. This is essential because CPaskal routine names must not collide with C standard library names that are already included by the runtime headers.


```
// WRONG -- "abs" collides with <cstdlib> abs
routine clink abs(const n: int32): int32; external "c";

// RIGHT -- "myabs" is unique, "name" maps to the real symbol
routine clink myabs(const n: int32): int32; external "c" name "abs";
```

External Variables

You can also declare external variables -- typically used to access globals exported by a DLL:

```
var dll_init_count: int32; external "bnf_dll_compliance";
```

 Linkage: clink vs cpplink

CPaskal supports two linkage modes that control how routine names are encoded in the compiled output:

Linkage	Convention	Name Mangling	Use Case
clink	C calling convention	None -- symbol name is used as-is	Calling C libraries, exporting from DLLs
cpplink	C++ calling convention	Itanium ABI mangling	Calling overloaded C++ functions

Most external declarations use `clink` because C libraries use unmangled names. Use `cpplink` only when you need to link against C++ symbols that use overloading (and therefore have mangled names).

```
// C linkage (most common)
routine clink myabs(const n: int32): int32; external "c" name "abs";

// C++ linkage (for overloaded C++ functions)
routine cpplink cpp_func(const x: float64): float64; external "mylib";
```

When you define a routine in a `dll` or `lib` module, use `clink` to ensure the exported symbol has a clean, unmangled name that other languages can call:

```
module dll mylib;

public routine clink my_func(const x: int32): int32;
begin
  return x * 2;
end;

end.
```

 Platform-Specific Externals

Different platforms provide different system libraries. Use conditional compilation to declare platform-specific external routines:

```
@ifdef TARGET_WIN64
routine clink GetTick(): uint64; external "kernel32" name "GetTickCount64";
@endif

#ifdef TARGET_LINUX64
routine clink ext_getpid(): int32; external "c" name "getpid";
#endif
```

 Note

`external "c"` links `libc` on both Windows and Linux through the Zig backend. You only need platform `@ifdef` guards for platform-specific libraries like `kernel32` on Windows.

The predefined conditional symbols for platform detection are:

Symbol	Platform
<code>TARGET_WIN64</code> , <code>WINDOWS</code> , <code>WIN64</code>	Windows x86_64
<code>TARGET_LINUX64</code> , <code>LINUX</code>	Linux x86_64

 DLL and Static Library Patterns

Creating a DLL

A `dll` module exports routines that other programs can call at runtime:

```
module dll mylib;

public routine clink my_func(const x: int32): int32;
begin
    return x * 2;
end;

public routine clink my_add(const a: int32; const b: int32): int32;
begin
    return a + b;
end;

initialize
    println("mylib loaded");
end;

finalize
    println("mylib unloaded");
end;

end.
```

Exported routines must be marked `public` and use `clink` for a clean C ABI. The `initialize` and `finalize` blocks run when the DLL is loaded and unloaded.

Consuming a DLL

From an `exe` module, declare the external routines and name the DLL:

```
module exe consumer;

routine clink my_func(const x: int32): int32; external "mylib";
routine clink my_add(const a: int32; const b: int32): int32; external "mylib";

begin
    println("my_func(21) = {}", my_func(21));
    println("my_add(10, 20) = {}", my_add(10, 20));
end.
```

The compiler probes for `mylib.lib` / `mylib.a` (static import library) first, then `mylib.dll` / `mylib.so` (dynamic library).

Creating a Static Library

A `lib` module produces a static library (`.lib` on Windows, `.a` on Linux) that is linked directly into the consumer's binary:


```
module lib mathutils;

public routine clink fast_add(const a: int32; const b: int32): int32;
begin
  return a + b;
end;

end.
```

Consuming a Static Library

Same pattern as DLLs, but you may need `@librarypath` to tell the compiler where to find the `.lib / .a` file:

```
module exe consumer;

@librarypath "$S:libs";

routine clink fast_add(const a: int32; const b: int32): int32; external "mathutils";

begin
  println("{} ", fast_add(10, 20));
end.
```



Tip

The `$S:` prefix resolves relative to the declaring module's directory. Use `$P:` for paths relative to the compiler executable, or `$D:` for the current working directory.

⚡ Inline C++ Code

CPaskal lets you inject raw C++ code directly into the generated output. This is an escape hatch for cases where you need C++ features that CPaskal doesn't expose, or need to include specific headers.

cppstart / cppend Blocks

Use `cppstart` and `cppend` to inject raw C++ into the generated header or source file:

```
cppstart header
#include <iostream>
#include <vector>
cppend

cppstart source
void my_cpp_helper() {
  std::cout << "Hello from raw C++" << std::endl;
}
cppend
```

The `header` or `source` keyword controls where the code is injected:

Target	Injected Into	Use Case
header	The generated <code>.h</code> file	<code>#include</code> directives, type declarations, extern declarations
source	The generated <code>.cpp</code> file	Function implementations, static data

cpp() Inline Expressions

For single C++ expressions, use `cpp()` anywhere an expression is expected:

```
var x: int32 = cpp("100 + 23");
cpp("std::cout << 'C' << std::endl");
```

The string argument is emitted verbatim as a C++ expression. No validation is performed on the content -- it is passed directly to the C++ compiler.

 Warning

Inline C++ bypasses all CPaskal type checking and safety guarantees. Use it sparingly and only when there is no CPaskal-native way to achieve what you need.

 Library Resolution

When the compiler encounters an `external` clause, it resolves the library name using the following search order:

Extension	Type	Behavior
<code>.lib / .a</code>	Static library	Linked directly into the binary
<code>.dll / .so</code>	Shared library	Loaded at runtime
<i>(none)</i>	Auto-probe	Tries static first, then shared

The compiler searches for libraries in:

- 1. The output directory
- 2. Paths added with `@librarypath`
- 3. System library paths (provided by Zig/Clang)

Use the `@addlinklibrary` directive to link additional libraries that aren't referenced by external clauses:

```
@addlinklibrary "SDL3";
@librarypath "$S:libs/SDL3";
```

 CImporter: Generating Bindings from C Headers

CPaskal includes CImporter, a tool that automatically generates `.cpas` binding files from C header files. This is how large C libraries like SDL3 and raylib are integrated.

Usage

```
cpas cimport <script.cis>
```

CImporter reads a `.cis` (CImporter Script) file that describes which headers to process and how to map them. The output is a `.cpas` unit module containing all the type declarations, constants, and external routine declarations needed to call the C library from CPaskal.

Script Commands

A `.cis` script is a sequence of commands that configure and run the import. Here is the full command reference:

Configuration:

Command	Description
<code>SetHeader("filename")</code>	C header file to import
<code>SetModuleName("name")</code>	Output module name
<code>SetDllName("name")</code>	DLL/shared library name

Command	Description
SetTargetDllName(target, "name")	DLL name for a specific target
SetDllPath("path")	Path to DLL for preprocessing
SetOutputPath("path")	Output directory for generated .cpas file
SetBindingMode(mode)	Binding mode (dynamic)
SetSavePreprocessed(bool)	Save preprocessor output (True / False)

Paths and filtering:

Command	Description
AddIncludePath("path" [, "module"])	C include search path
AddSourcePath("path")	C source search path
AddExcludedType("name")	Skip a C type during import
AddExcludedFunction("name")	Skip a C function during import
AddFunctionRename("original", "newname")	Rename an imported function
AddUsesUnit("unit")	Add a module dependency (import)
AddDefine("name" [, "value"])	Preprocessor #define
AddUndefine("name")	Preprocessor #undef

Text manipulation (post-process the generated output):

Command	Description
InsertTextAfter("target", "text" [, occurrence])	Insert text after a match
InsertTextBefore("target", "text" [, occurrence])	Insert text before a match
InsertFileAfter("target", "file" [, occurrence])	Insert file contents after a match
InsertFileBefore("target", "file" [, occurrence])	Insert file contents before a match
ReplaceText("old", "new" [, occurrence])	Replace text in the generated output

Linking:

Command	Description
AddCopyDll(target, "path")	Copy DLL/SO to output during build
AddLinkLibrary(target, "path")	Add library search path for target
AddDllNameMap("path", "dllname", "dllpath")	DLL name mapping
AddDepDll("dllname", "dllpath")	Dependency DLL

Execution:

Command	Description
Process()	Run the import with current settings
Clear()	Reset all settings

Enum values used in commands:

Category	Values
target	x86_64_windows , x86_64_linux
mode	dynamic
bool	True , False

Example: Importing raylib

text

```
SetSavePreprocessed(False);
SetBindingMode(dynamic);
SetModuleName("raylib");
SetDllName("raylib");
SetOutputPath("res\\libs\\vendor\\raylib");
SetDllPath("res\\libs\\vendor\\raylib\\win64\\raylib.dll");
AddIncludePath("res\\libs\\vendor\\raylib\\include");
AddSourcePath("res\\libs\\vendor\\raylib\\include");
AddExcludedType("va_list");
SetHeader("res\\libs\\vendor\\raylib\\include\\raylib.h");
AddCopyDll(x86_64_windows, "$P:res/libs/vendor/raylib/win64/raylib.dll");
AddCopyDll(x86_64_linux, "$P:res/libs/vendor/raylib/linux64/libraylib.so.550");
AddLinkLibrary(x86_64_linux, "$P:res/libs/vendor/raylib/linux64/");
Process();
```

This script tells Clmporter to parse `raylib.h`, generate a `raylib.cpas` module, exclude the unsupported `va_list` type, and set up DLL copying for both Windows and Linux targets.

Example: Importing SDL3 with a Dependency

When a library depends on another (e.g., `SDL3_mixer` depends on `SDL3`), use `AddUsesUnit` to declare the dependency:

text

```
SetSavePreprocessed(False);
SetBindingMode(dynamic);
SetModuleName("SDL3_mixer");
SetDllName("SDL3_mixer");
SetOutputPath("res\\libs\\vendor\\sdl3_mixer");
SetDllPath("res\\libs\\vendor\\sdl3_mixer\\win64\\SDL3_mixer.dll");
AddIncludePath("res\\libs\\vendor\\sdl3\\include");
AddSourcePath("res\\libs\\vendor\\sdl3_mixer\\include\\SDL3");
AddUsesUnit("SDL3");
AddExcludedType("va_list");
SetHeader("res\\libs\\vendor\\sdl3_mixer\\include\\SDL3\\SDL_mixer.h");
AddCopyDll(x86_64_windows, "$P:res/libs/vendor/sdl3_mixer/win64/SDL3_mixer.dll");
Process();
```

The `AddUsesUnit("SDL3")` ensures the generated module imports `SDL3` and uses module-qualified references for `SDL3` types.

 **Tip**

🔦 Notice that `AddIncludePath` and `AddSourcePath` point to different folders. `AddSourcePath` tells Clmporter which headers to actually import -- only the mixer headers in this case. `AddIncludePath` makes the `SDL3` headers visible for type resolution, but since they live in a separate folder, Clmporter knows not to pull those types into the generated binding. This separation is what prevents the dependency's types from being duplicated -- they stay in the `SDL3` module where they belong.

Using the Generated Bindings

Once Clmporter has generated the bindings, using them from CPaskal is straightforward:

```

module exe sdl_demo;

import sdl3;

begin
  sdl3.SDL_Init(sdl3.SDL_INIT_VIDEO);
  // ... SDL3 code ...
  sdl3.SDL_Quit();
end.

```

All imported symbols are accessed through the module qualifier, keeping the namespace clean.

Calling Conventions and ABI

CPaskal's C interop is built on these ABI guarantees:

- All 16 primitive types map directly to C/C++ types with no conversion overhead (see [Language Reference](#) for the full mapping table)
- Records map to C structs with identical memory layout (use `record packed` for packed structs)
- Pointers are raw machine pointers -- `pointer to int32` is `int32_t*`
- Strings (`string` / `wstring`) are C++ `std::string` / `std::wstring` -- pass `cstr(s)` to get a raw `char*` for C APIs, or `wstr(s)` for `wchar_t*`
- The `clink` linkage produces symbols with no name mangling, compatible with any C-linkage consumer

Tip

 When passing CPaskal strings to C functions that expect `char*`, use the `cstr()` intrinsic: `my_c_func(cstr(my_string))`. The returned pointer borrows into the managed string's storage -- no allocation or copy.

Complete Interop Example

Here is a complete example that demonstrates calling libc functions from CPaskal with platform-specific externals:

```

module exe interop_demo;

// C standard library (works on both Windows and Linux)
routine clink myabs(const n: int32): int32; external "c" name "abs";
routine clink mytoupper(const c: int32): int32; external "c" name "toupper";

// Platform-specific externals
#ifdef TARGET_WIN64
routine clink GetTick(): uint64; external "kernel32" name "GetTickCount64";
#endif

begin
  println("abs(-42) = {}", myabs(-42));
  println("toupper('a') = {}", int32(mytoupper(int32("a"))));

  #ifdef TARGET_WIN64
  println("tick count = {}", GetTick());
  #endif
end.

```

This program:

1. Declares two libc wrappers (`myabs` , `mytoupper`) that work on all platforms
2. Declares a Windows-only system call (`GetTick`) guarded by conditional compilation
3. Calls all of them from the main body, with the Windows call also conditionally compiled

Memory & Data Structures

CPaskal gives you direct control over memory layout and data organization. From stack-allocated records and fixed-size arrays to heap-allocated pointers and dynamic arrays, you choose exactly how your data lives in memory. Because CPaskal compiles to C++23, these constructs map directly to their native equivalents with zero abstraction overhead.

Records

Records are CPaskal's primary composite data type -- equivalent to C structs. They group related fields into a single named type with a predictable memory layout.

```
type
  Point = record
    x: int32;
    y: int32;
  end;
```

Access fields with dot notation:

```
var p: Point;
p.x := 42;
p.y := 99;
println("{}, {}", p.x, p.y);
```

Record Literals

You can initialize a record in a single expression using named field syntax:

```
var p: Point = Point(x: 42, y: 99);
var person: Person = Person(name: "Alice", age: 30);
```

Nested record literals work too:

```
var line: Line = Line(start: Point(x: 1, y: 2), finish: Point(x: 10, y: 20));
```

Packed Records

By default, the compiler may insert padding between fields for alignment. Use `packed` to eliminate all padding and lay fields out contiguously:

```
type
  PackedPair = record packed
    a: int8;
    b: int32;
  end;
```

Note

Packed records use less memory but may have slower field access on some architectures due to misaligned reads. Use them when you need exact binary layout -- for file formats, network protocols, or hardware registers.

Aligned Records

Force a specific alignment for the entire record:


```

type
  Aligned32 = record align(32)
    val: int32;
  end;

```

The alignment value must be a power of two. This is useful for SIMD data, cache-line optimization, or interfacing with hardware that requires specific alignment guarantees.

Record Inheritance

Records can derive from a base record. The derived record contains all fields of the base, followed by its own:

```

type
  Base = record
    id: int32;
  end;

  Derived = record(Base)
    extra: int32;
  end;

```

A `Derived` value has both `id` and `extra` fields. The base fields come first in memory layout, so a pointer to `Derived` is layout-compatible with a pointer to `Base`.

Bit Fields

Records can declare fields with explicit bit widths for compact binary packing:

```

type
  BitPack = record
    a: uint32 : 4;    // 4-bit field (values 0..15)
    b: uint32 : 4;    // another 4-bit field
    c: uint32 : 8;    // 8-bit field (values 0..255)
  end;

```

Bit fields are useful for hardware register mappings, compact flags, and binary protocol fields where every bit matters.

Overlays (Unions)

An overlay lets multiple fields share the same memory -- only one is valid at a time. This is equivalent to a C `union`:

```

type
  Value = overlay
    asInt: int64;
    asFloat: float64;
  end;

```

Both `asInt` and `asFloat` occupy the same 8 bytes. Writing to one and reading the other reinterprets the raw bits.

Anonymous Overlays in Records

You can embed an anonymous overlay directly inside a record to create tagged union patterns:

```

type
  Tagged = record
    tag: int32;
    overlay
      iVal: int64;
      fVal: float64;
    end;
  end;

```

The `tag` field sits at its own offset, while `iVal` and `fVal` share the same memory after it. Your code uses `tag` to know which overlay field is currently valid.

Choices (Enumerations)

Choices define a set of named integer constants -- equivalent to C enums:

```
type
  Color = choices(red, green, blue = 5, alpha);
```

Values are sequential starting from 0 unless explicitly assigned. After `blue = 5`, `alpha` becomes 6.

Access choice values with dot notation:

```
var c: Color = Color.red;

match c of
  Color.red:  println("red");
  Color.green: println("green");
  Color.blue:  println("blue");
  Color.alpha: println("alpha");
end;
```

Tip

 Choices are always qualified with their type name (`Color.red` , not just `red`). This prevents name collisions when multiple choice types define similar values.

Arrays

Fixed-Size Arrays

Fixed-size arrays have bounds known at compile time:

```
var arr: array[0..4] of int32;
arr[0] := 10;
arr[1] := 20;
println("first = {}", arr[0]);
```

The bounds are inclusive -- `array[0..4]` has 5 elements (indices 0 through 4).

Dynamic Arrays

Dynamic arrays are heap-allocated and can be resized at runtime:

```
var darr: array of int32;
setlength(darr, 5);          // allocate 5 elements
darr[0] := 100;
darr[1] := 200;
println("len = {}", len(darr)); // prints 5
```

Use `setlength` to allocate or resize, and `len` to query the current length. Elements are zero-indexed.

Note

Dynamic arrays are managed -- they are automatically freed when they go out of scope. You do not need to manually free them.

Pointers

Pointers hold the memory address of another value. CPaskal supports both typed and untyped pointers.

Typed Pointers

A typed pointer knows what type it points to:

```
var x: int32 = 42;
var p: pointer to int32 = address of x;
println("{} ", p^);    // dereference: prints 42
```

Operator	Meaning
<code>address of expr</code>	Get the address of a variable or field
<code>expr^</code>	Dereference a pointer (access the value it points to)

You can declare pointer types for reuse:

```
type
  IntPtr = pointer to int32;
  ConstPtr = pointer to const int32;
```

A `pointer to const` prevents modification through the pointer -- the pointed-to value is read-only.

Untyped Pointers

The bare `pointer` type is an untyped pointer, equivalent to C's `void*`:

```
var p: pointer;
```

Untyped pointers cannot be dereferenced directly -- you must cast them to a typed pointer first. They are useful for raw memory operations and interop with C APIs that use `void*`.

nil

The `nil` literal represents a null pointer:

```
var p: pointer to int32 = nil;
if p = nil then
  println("null pointer");
end;
```

Heap Allocation

CPaskal provides two levels of heap memory management: typed allocation with `new` / `dispose`, and raw allocation with `getmem` / `freemem` / `resizemem`.

new / dispose (Typed Allocation)

`new` allocates memory for a typed pointer and default-constructs the value. `dispose` frees it and sets the pointer to `nil`:

```
var p: pointer to Point;
new(p);           // allocate + default-construct
p^.x := 42;
p^.y := 99;
println("{} {}", p^.x, p^.y);
dispose(p);       // free + set to nil
```

💡 Tip

💡 Always pair `new` with `dispose`. After `dispose`, the pointer is automatically set to `nil`, so accidental double-frees are harmless.

getmem / freemem / resizemem (Raw Allocation)

For raw, unstructured memory -- buffers, byte arrays, interop with C APIs:

```
var buf: pointer to uint8;
getmem(buf);           // allocate
resizemem(buf, 1024);  // resize to 1024 bytes
// ... use buf ...
freemem(buf);          // free
```

Intrinsic	Purpose
getmem(ptr)	Allocate raw memory
freemem(ptr)	Free raw memory
resizemem(ptr, size)	Resize an existing allocation

📄 Note

Raw memory intrinsics do not construct or destruct values. The memory is uninitialized after `getmem`. Use `new` / `dispose` when you need proper initialization.

🎲 Sets

Sets represent collections of integer values, implemented as efficient bitmasks. They support up to 64 elements with an arbitrary base offset.

Creating Sets

```
var s: set;
s := [1, 3, 5, 7];      // individual elements
s := [1..10];           // range
s := [1, 3..7, 10];     // mixed elements and ranges
```

Set Type Declarations

```
type SmallSet = set of 0..31;
```

Membership Test

```
if 5 in s then
  println("5 is in the set");
end;
```

Set Operations

Operator	Operation	Example
+	Union	var u: set = a + b;
*	Intersection	var i: set = a * b;

Operator	Operation	Example
-	Difference	var d: set = a - b;
=	Equality	if a = b then ...
<>	Inequality	if a <> b then ...
in	Membership	if x in s then ...

```
var odds: set = [1, 3, 5, 7, 9];
var primes: set = [2, 3, 5, 7];

var both: set = odds * primes;      // intersection: [3, 5, 7]
var either: set = odds + primes;    // union: [1, 2, 3, 5, 7, 9]
var oddOnly: set = odds - primes;   // difference: [1, 9]
```

 Tip

 Sets are stack-allocated bitmasks, not heap collections. Operations like union, intersection, and membership test are single CPU instructions. Use them freely for flags, permissions, feature toggles, and any situation where you need fast membership testing over a bounded range of integers.

 Forward Declarations

When two types need to reference each other, or when you want to declare a routine before defining it, use forward declarations:

Forward Types

```
forward type MyRecord;

type
  MyPtr = pointer to MyRecord;  // OK -- pointer to forward-declared type

// ... later in the same module:
type
  MyRecord = record
    val: int32;
    next: MyPtr;
  end;
```

 Note

Forward-declared types can only be used in `pointer to` contexts until the full declaration appears. You cannot declare variables of a forward type or access its fields until it is fully defined.

Forward Routines

```
forward routine compute(a: int32): int32;

// ... can call compute() here ...

routine compute(a: int32): int32;
begin
  return a * 2;
end;
```

The forward declaration and the full definition must have matching signatures. Unresolved forwards at module end are compile errors.

Type Casts

Explicit type casts convert values between compatible types:

```
var x: int64 = 12345;
var y: int32 = int32(x);           // narrow: int64 -> int32

var f: float64 = 3.14;
var i: int32 = int32(f);           // truncate: float64 -> int32
```

The cast syntax uses the target type as a function call: `TargetType(expr)` . The compiler validates that the conversion is meaningful -- you cannot cast between completely unrelated types.

Summary

Feature	Stack	Heap	Managed
Records	✓	Via <code>new</code> / <code>dispose</code>	No
Fixed arrays	✓	No	No
Dynamic arrays	No	✓	Yes (auto-freed)
Typed pointers	✓ (the pointer itself)	Points to heap with <code>new</code>	No
Sets	✓	No	No
Choices	✓	No	No

BNF Grammar

Syntax Notation

This section is the formal grammar reference for the CPaskal Programming Language. It is intended for implementers, tooling authors, and anyone who needs exact syntax rules. For an easier language walkthrough, see [Language Reference](#).

The grammar uses EBNF notation. Brackets `[` and `]` mark optional elements. Braces `{` and `}` mark repetition, zero or more times. Parentheses group alternatives. The vertical bar `|` separates alternatives. Terminal symbols are enclosed in quotes or written as lowercase literal tokens. Non-terminals are written in PascalCase.

 **Note**

 This file is intentionally formal. Use it when you need the exact grammar contract. Use the [Language Reference](#) for explanations and the [Common Tasks](#) for examples.

How to Read This Grammar

Symbol	Meaning
<code>A B</code>	<code>A</code> followed by <code>B</code>
<code>A B</code>	either <code>A</code> or <code>B</code>
<code>[A]</code>	optional <code>A</code>
<code>{ A }</code>	zero or more repetitions of <code>A</code>
<code>(A B)</code>	grouped alternatives
<code>"text"</code>	literal source text

 Tip

 When implementing a parser, treat this file as the external behavior contract, not as a required internal parser architecture. Recursive descent, Pratt parsing, table-driven parsing, or another strategy can all implement the same grammar.

 1. Lexical Elements

```
letter      = "A" | ... | "Z" | "a" | ... | "z" | "_" .
digit       = "0" | ... | "9" .
hexDigit    = digit | "A" | ... | "F" | "a" | ... | "f" .
character   = (* any source character except the delimiter *) .
newline     = (* line feed (U+000A) *) .

ident       = letter { letter | digit } .
integer     = digit { digit } | "0" ( "x" | "X" ) hexDigit { hexDigit } .
float_literal = digit { digit } "." { digit } [ exponent ] [ "f" | "F" ] .
exponent    = ( "e" | "E" ) [ "+" | "-" ] digit { digit } .
cstring     = '"' { character | escapeSeq } '"' .
wstring     = "w" '"' { character | escapeSeq } '"' .
escapeSeq   = "\" ( "n" | "t" | "r" | "0" | "\" | "'" | '"' | "x" hexDigit hexDigit ) .
```

text

 Numeric Literal Type Rules

Literal	Suffix	Type	Example
42	--	int32	integer
1.5	--	contextual	float literal
1.5f, 1.5F	f / F	float32	explicit float32

Float literal resolution without a suffix:

- Assigned to a float32 variable or passed to a float32 parameter: float32
- Assigned to a float64 variable or passed to a float64 parameter: float64
- Ambiguous or unknown context: float64

Float literal resolution with f or F suffix:

- Always float32, regardless of context

 String Literal Convention

- "..." -- String literal. Escape sequences processed. UTF-8 encoded.
- w"..." -- Wide string literal. Escape sequences processed. UTF-16 encoded. Prefix is case-sensitive: only lowercase w.

 Character Type Assignment Rules

The char and wchar types have no dedicated literal syntax. Characters are assigned using string literals, variable-to-variable assignment, or string indexing. The semantic pass validates type compatibility using the AST.

Valid char assignments:

- c := "x"; -- A cstring literal of exactly one character. The semantic pass verifies len = 1; longer literals produce a compile error.
- c := d; -- Where d is also of type char.
- c := s[i]; -- Indexing a string yields a char.

Valid wchar assignments:

- `wc := w"x";` -- A `wstring` literal of exactly one character (semantic-checked).
- `wc := wd;` -- Where `wd` is also of type `wchar` .
- `wc := ws[i];` -- Indexing a `wstring` yields a `wchar` .

Invalid assignments (compile error):

- `c := "abc";` -- Multi-character literal assigned to `char` .
- `c := s;` -- `string` variable assigned to `char` (use indexing instead).
- `c := wc;` -- `wchar` assigned to `char` (width mismatch).
- `wc := c;` -- `char` assigned to `wchar` (width mismatch).

🚫 2. Reserved Words

The language is **case-sensitive** for keywords and identifiers.

text

address	align	and	array	assert	asserteq
asserteqf	assertfalse	assertfail	assertnil	assertnotnil	asserttrue
begin	break	choices	clink	const	continue
cpplink	cppstart	cpend	cpp	cstr	dispose
div	do	downto	else	end	except
exccode	excmsg	external	false	finalize	finally
for	forward	freemem	getmem	guard	
if	import	in	initialize	is	len
match	mod	module	new		
nil	not	of	or	overlay	
packed	paramcount	paramstr	pointer	print	
println	public	record	repeat	resizemem	
return	routine	set	setlength	shl	
shr	size	test	then	throw	
throwcode	to	true	type	until	utf8
var	varargs	while	wstr	xor	

📌 Note

The identifiers `exe` , `dll` , `lib` , and `unit` are contextual. They have special meaning only in the `ModuleKind` position and may be used as ordinary identifiers elsewhere. Unit modules are `.cpas` source files that are compiled inline into the importing module rather than producing separate output.

🧱 3. Built-in Types

text

int8	int16	int32	int64
uint8	uint16	uint32	uint64
float32	float64		
boolean			
char	wchar		
string	wstring		
pointer			

📏 Type Sizes

Type	Size (bytes)	Description
int8	1	Signed 8-bit integer
int16	2	Signed 16-bit integer
int32	4	Signed 32-bit integer
int64	8	Signed 64-bit integer

Type	Size (bytes)	Description
uint8	1	Unsigned 8-bit integer
uint16	2	Unsigned 16-bit integer
uint32	4	Unsigned 32-bit integer
uint64	8	Unsigned 64-bit integer
float32	4	32-bit IEEE 754 float
float64	8	64-bit IEEE 754 float
boolean	1	Boolean (0 or 1)
char	1	8-bit character
wchar	2	16-bit wide character
string	8 (pointer)	Managed UTF-8 string
wstring	8 (pointer)	Managed UTF-16 string
pointer	8	Untyped pointer

⚙️ 4. Operators and Delimiters

+	-	*	/	=	<>	<	>	<=	>=
:=	+=	-=	*=	/=					
:	;	,	.	..	...	^		&	
(	)	[	]						

text

💡 Operator Semantics

- `:=` -- Assignment
- `=` -- Equality comparison
- `<>` -- Not equal
- `^` -- Postfix: pointer dereference
- `&` -- Prefix: address-of (see also `address of`)
- `|` -- Reserved token (available for future use)

💬 5. Comments

Comment	= <code>"/" { character } newline</code> <code>"/" { character Comment } "*" .</code>
---------	--

text

- `//` -- Line comment.
- `/* ... */` -- Block comment. May be nested.

📌 Note

`( * )` and `{ }` are not comment delimiters in CPaskal.

6. Module Structure

text

```

Module      = "module" ModuleKind ident ";" [ Directives ] [ ImportClause ]
             { Declaration }
             [ "initialize" StatementSeq "end" ";" ]
             [ "finalize" StatementSeq "end" ";" ]
             ( MainBody | "end" "." )
             { TestBlock } .

MainBody     = "begin" StatementSeq "end" "." .    (* exe modules only *)

ModuleKind   = "exe" | "dll" | "lib" | "unit" .

Directives   = { Directive } .
Directive    = "@" ident [ DirectiveValue ] ";" .
DirectiveValue = cstring | integer | float_literal | ident .

ImportClause = "import" ident { "," ident } ";" .

TestBlock    = "test" cstring [ "var" { VarDecl } ]
             "begin" StatementSeq "end" ";" .

```

Note

Module lifecycle: `initialize` and `finalize`. The `initialize` and `finalize` blocks are module lifecycle hooks. `initialize` runs at startup (before the entry point), `finalize` runs at shutdown. Both are optional and supported on all module kinds. They are separate from `begin`, which is the main program body for exe/dll modules. For unit modules, `initialize` / `finalize` replace the old `begin` / `finalize` embedded syntax. The SSA pass auto-discovers these functions by name prefix and wires them into the entry point.

Important

Module qualification rule. All public symbols from an imported module must be accessed using full module qualification: `moduleName.symbolName`. Unqualified access to imported symbols is a compile error. This applies to routines, types, variables, and constants alike. If modules A and B both export a symbol `Foo`, they are distinguished as `A.Foo` and `B.Foo` -- there is no ambiguity.

Note

Directive termination. Every directive is terminated by `;` -- with one exception: the seven conditional-compilation directives (Section 7: `@define`, `@undef`, `@ifdef`, `@ifndef`, `@elseif`, `@else`, `@endif`) take **no** terminator.

Note

Test blocks. Test blocks appear after `end.` and are only compiled when `@unittestmode on;` is active. Each test block has a string name, optional local variables, and a body. When unittest mode is on, the compiler replaces the normal entry point with the test runner. Test blocks have access to all module declarations.

7. Conditional Compilation

```
ConditionalDirective = DefineDir | UndefDir | IfdefDir | IfndefDir
                    | ElseIfDir | ElseDir | EndifDir .
```

text

```
DefineDir    = "@define" ident .
UndefDir     = "@undef" ident .
IfdefDir     = "@ifdef" ident .
IfndefDir    = "@ifndef" ident .
ElseIfDir    = "@elseif" ident .
ElseDir      = "@else" .
EndifDir     = "@endif" .
```

Directives

All directives below are terminated by `;`. Bare identifiers are the canonical form for enumerated values; quoted strings are reserved for paths and free text.

Module-level directives (appear after `module` header, before or among declarations):

- `@exeicon "path";` -- Sets the application icon (Windows EXE modules only).
- `@resfile "path";` -- Specifies a compiled resource file (.res) to link into the output.
- `@outputpath "path";` -- Sets the output directory for the compiled binary.
- `@copydll "path";` -- Copies a DLL/shared library to the output directory during build.
- `@addlinklibrary "path";` -- Links an additional static or shared library into the output.
- `@librarypath "path";` -- Adds a directory to the library and module search path.
- `@modulepath "path";` -- Adds a directory to the module (unit) search path.
- `@includepath "path";` -- Adds a directory to the include search path.
- `@subsystem console|gui;` -- Sets the application subsystem (bare identifier). Default: `console`. Windows-only: on the linux64 target it produces a warning and is ignored.
- `@target win64|linux64;` -- Sets the compilation target (bare identifier). Default: `win64`. Overrides the API SetTarget for the current compile only; must appear in the root module.
- `@optimize debug|none|basic|full;` -- Sets optimization level (bare identifier).
- `@unittestmode on|off;` -- Enables or disables test block compilation and test runner entry point (bare identifier).

Path resolution

Every directive taking a `"path"` resolves it the same way. An absolute path is used as-is. A relative path resolves against the directory of the module that declares the directive, so a module and the files it references travel together.

An optional prefix overrides that base:

Prefix	Base	Use for
<code>\$P:</code>	Directory of the running compiler executable	Shipped assets under the compiler's own <code>res</code> tree
<code>\$D:</code>	Current working directory	Paths relative to where the compiler was invoked
<code>\$S:</code>	Declaring module's directory -- the default, stated explicitly	Clarity in modules that mix bases

The prefix is matched case-insensitively and only at the very start of the string. A path containing `$P:` anywhere else is left alone.

`$P:` is the correct choice for any module meant to be imported from another folder. A vendor binding that says `@copydll "res/libs/vendor/raylib/win64/raylib.dll"` resolves against its own directory and fails as soon as the module is used from elsewhere; the `$P:` form always finds the file shipped beside the compiler:

```
@copydll "$P:res/libs/vendor/raylib/win64/raylib.dll";
@libpath "$P:res/libs/vendor/raylib";
@exeicon "$P:res/assets/icons/cpaskal.ico";
```

Version information directives (for embedding in the PE executable):

- `@addverinfo on|off;` -- Enables or disables version information embedding (bare identifier).
- `@vimajor number;` -- Major version number.
- `@viminor number;` -- Minor version number.
- `@vipatch number;` -- Patch version number.
- `@viproductname "name";` -- Product name.
- `@videscription "text";` -- File description.
- `@vifilename "name";` -- Original filename.
- `@vicompanyname "name";` -- Company name.
- `@vicopyright "text";` -- Copyright string.

Statement-level directives:

- `@breakpoint;` -- Marks a debugger breakpoint location. Takes no value.
- `@message hint|warn|error|fatal "text";` -- Emits a compiler diagnostic at parse time (bare-identifier severity followed by a quoted string).

 Note

Conditionals in imported units. The conditional-compilation directives (`@define`, `@undef`, `@ifdef`, `@ifndef`, `@elseif`, `@else`, `@endif`) take no terminator and also work inside imported unit modules, evaluated with the root module's defines (e.g. `TARGET_WIN64`).

Predefined Symbols

Symbol	Defined when
CPASKAL	Always
CPUX64	Always (x64-only architecture)
APPTYPE_CONSOLE	Always
WINDOWS	Target is win64
MSWINDOWS	Target is win64
WIN64	Target is win64
TARGET_WIN64	Target is win64
LINUX	Target is linux64
TARGET_LINUX64	Target is linux64
DEBUG	Optimization level is none
RELEASE	Optimization level is not none
BUILD_EXE	Module kind is exe (or unknown)
BUILD_DLL	Module kind is dll
BUILD_LIB	Module kind is lib

8. Declarations

```

Declaration      = [ "public" ] ( ConstSection | TypeSection | VarSection | RoutineDecl )
                  | ForwardDecl .

ConstSection     = "const" { [ "public" ] ConstDecl } .
ConstDecl        = ident [ ":" TypeExpr ] "=" Expression ";" .

TypeSection      = "type" { [ "public" ] TypeDecl } .
TypeDecl         = ident "=" TypeDef ";" .

VarSection       = "var" { [ "public" ] VarDecl } .
VarDecl          = ident ":" TypeExpr [ "=" Expression ] ";" [ ExternalVarClause ] .
ExternalVarClause = "external" [ cstring | ident ] [ "name" cstring ] ";" .

ForwardDecl      = "forward" ( ForwardType | ForwardRoutine ) .
ForwardType      = "type" ident ";" .
ForwardRoutine   = "routine" [ LinkageSpec ] ident [ FormalParams ] [ ":" TypeExpr ] ";" .

```

⚠ Important

Forward declaration semantics. A `forward` declaration introduces a name before its full definition appears. The full declaration must appear later in the same module. For types, a forward-declared name may only be used in `pointer to` contexts until the full definition is seen, because the type's size and layout are unknown. For routines, the forward carries the full signature, so calls are valid immediately. If a forward declaration has no matching full declaration by module end, it is a compile error. The full declaration must match the forward exactly (same signature for routines, same kind for types).

🔧 9. Routine Declarations

```

RoutineDecl      = "routine" [ LinkageSpec ] ident [ FormalParams ] [ ":" TypeExpr ] ";"
                  ( ExternalClause | RoutineBody ) .

LinkageSpec       = "clink" | "cpplink" .

FormalParams      = "(" [ ParamList ] ")" .
ParamList         = ParamDecl { ";" ParamDecl } [ ";" "..." ] | "..." .
ParamDecl         = [ "var" | "const" ] ident ":" TypeExpr .

ExternalClause    = "external" [ cstring | ident ] [ "name" cstring ] ";" .

RoutineBody       = [ "type" { TypeDecl } ]
                  [ "const" { ConstDecl } ]
                  [ "var" { VarDecl } ]
                  "begin" StatementSeq "end" ";" .

```

- **C linkage** (`clink`): Explicit C calling convention and naming. This is also the default when no linkage spec is given.
- **C++ linkage** (`cpplink`): Enables Itanium ABI name mangling for C++ interoperability and overloading.

🔄 Routine Overloading

Multiple routines with the same name but different parameter signatures are permitted under `cpplink` linkage. Overload resolution at the call site matches by argument count and parameter types.

text

Overloading requires `cpplink`. If overloaded routines are declared without `cpplink` (i.e., with `clink` or no linkage spec), the compiler auto-promotes them to `cpplink` and emits a warning. This ensures correct C++ name mangling in the generated code.

```
// Explicit cpplink
routine cpplink add(a: int32; b: int32): int32;
routine cpplink add(a: float64; b: float64): float64;

// Implicit promotion (compiler warns, defaults to cpplink)
routine add(a: int32; b: int32): int32;
routine add(a: int32; b: int32; c: int32): int32;
```

text

External Clause Semantics

The optional value after `external` names the library to import from:

- **String literal** -- the library name/path directly: `external "raylib.dll";`
- **Identifier** -- names a module-level string constant declared in the enclosing module; the constant's value is used as the library name. A compile error is raised if no such string constant exists.

```
public const DLL_NAME: string = "raylib";

routine InitWindow(const width: int32; const height: int32;
  const title: pointer); external DLL_NAME;
```

text

Extension resolution rules for the library name:

- `.lib` / `.a` -- static import library.
- `.dll` / `.so` / `.so.<version>` -- dynamic import.
- **Extensionless** -- the library search paths are probed for a static library first; if found, static import. Otherwise dynamic: on linux64 the search paths are probed for `lib<name>.so.<version>`, `lib<name>.so`, then `<name>.so` (the found filename becomes the runtime dependency); if no probe hits, the target's default shared-library extension is appended.

10. Type Definitions

text

```

TypeDef      = RecordType | OverlayType | ArrayType
              | PointerType | SetType | ChoicesType | RoutineType | TypeExpr .

RecordType   = "record" [ "packed" ] [ "align" "(" integer ")" ]
              [ "(" TypeExpr ")" ]
              { FieldDecl | AnonOverlay } "end" .

OverlayType  = "overlay" { FieldDecl | AnonRecord } "end" .
AnonRecord   = "record" [ "packed" ] { FieldDecl | AnonOverlay } "end" ";" .
AnonOverlay  = "overlay" { FieldDecl | AnonRecord } "end" ";" .

FieldDecl    = ident ":" TypeExpr [ ":" integer ] ";" .

ArrayType    = "array" [ "[" [ ArrayBounds ] "]" ] "of" TypeExpr .
ArrayBounds  = integer ".." integer .

PointerType  = "pointer" [ "to" [ "const" ] TypeExpr ] .

SetType      = "set" [ "of" ( integer ".." integer | TypeExpr ) ] .

ChoicesType  = "choices" "(" ChoicesValue { "," ChoicesValue } ")" .
ChoicesValue = ident [ "=" Expression ] .

RoutineType  = "routine" [ LinkageSpec ] "(" [ ParamList ] ")" [ ":" TypeExpr ] .

TypeExpr     = QualIdent
              | "pointer" [ "to" [ "const" ] TypeExpr ]
              | "array" [ "[" [ ArrayBounds ] "]" ] "of" TypeExpr
              | "set" [ "of" ( integer ".." integer | TypeExpr ) ] .

QualIdent    = ident { "." ident } .

```

Note

`choices` is used instead of `enum`,
 and `overlay` instead of `union`. Anonymous overlays and records can nest
 inside each other for C data interop. Records support single inheritance
 via `record(BaseType)` syntax and bit fields via `fieldname: type : width`.

11. Statements

text

```

StatementSeq    = { Statement } .

Statement       = [ Assignment | CallStmt | IfStmt | WhileStmt | ForStmt
                  | RepeatStmt | BreakStmt | ContinueStmt
                  | MatchStmt | ReturnStmt | GuardStmt | RaiseStmt
                  | NewStmt | DisposeStmt
                  | GetMemStmt | FreeMemStmt | ResizeMemStmt | SetLengthStmt
                  | PrintStmt
                  | AssertStmt | InlineVarDecl | Directive | ";" ] .

InlineVarDecl   = "var" ident ":" TypeExpr [ "=" Expression ] ";" .

Assignment      = Designator ( "!=" | "+=" | "-=" | "*=" | "/=" ) Expression [ ";" ] .

CallStmt        = Designator [ ";" ] .

IfStmt          = "if" Expression "then" StatementSeq [ "else" StatementSeq ] "end" [ ";" ] .

WhileStmt       = "while" Expression "do" StatementSeq "end" [ ";" ] .

ForStmt         = "for" ident ":@" Expression ( "to" | "downto" ) Expression
                  "do" StatementSeq "end" [ ";" ] .

RepeatStmt      = "repeat" StatementSeq "until" Expression [ ";" ] .

BreakStmt       = "break" [ ";" ] .
ContinueStmt    = "continue" [ ";" ] .

MatchStmt       = "match" Expression "of" { MatchArm } [ "else" StatementSeq ] "end" [ ";" ] .
MatchArm        = MatchLabel { "," MatchLabel } ":" StatementSeq .
MatchLabel      = Expression [ ".." Expression ] .

ReturnStmt      = "return" [ Expression ] [ ";" ] .

GuardStmt       = "guard" StatementSeq
                  ( "except" StatementSeq [ "finally" StatementSeq ]
                  | "finally" StatementSeq ) "end" [ ";" ] .

RaiseStmt       = ( "throw" "(" Expression ")"
                  | "throwcode" "(" Expression "," Expression ")" ) [ ";" ] .

NewStmt         = "new" "(" Expression ")" [ ";" ] .
DisposeStmt     = "dispose" "(" Expression ")" [ ";" ] .
GetMemStmt      = "getmem" "(" Expression ")" [ ";" ] .
FreeMemStmt     = "freemem" "(" Expression ")" [ ";" ] .
ResizeMemStmt   = "resizemem" "(" Expression "," Expression ")" [ ";" ] .
SetLengthStmt   = "setlength" "(" Expression "," Expression ")" [ ";" ] .
PrintStmt       = ( "print" | "println" ) "(" [ ArgList ] ")" [ ";" ] .

```

Note

`break` and `continue` are valid only inside a `while`, `for`, or `repeat` body (compile error SEM008 otherwise). `break` exits the innermost loop; `continue` starts its next iteration. In a `for` loop, `continue` still performs the iterator step before re-testing the bound.

Assert Statements (Unit Testing)

Assert statements are available in all code but are primarily used inside test blocks.

All assertions continue after failure -- failures accumulate and are reported per test.

The compiler handles all test infrastructure automatically. When `@unittestmode on`; is active, test blocks are compiled, registered, and executed by the built-in test runner.

The compiler injects source file and line number automatically.


```
AssertStmt      = ( "assert" "(" Expression ")"
                  | "asserttrue" "(" Expression ")"
                  | "assertfalse" "(" Expression ")"
                  | "asserteq" "(" Expression "," Expression ")"
                  | "asserteqf" "(" Expression "," Expression "," Expression ")"
                  | "assertnil" "(" Expression ")"
                  | "assertnotnil" "(" Expression ")"
                  | "assertfail" "(" Expression ")" ) [ ";" ] .
```

- `assert(expr)` -- Fails if `expr` is false.
- `asserttrue(expr)` -- Fails if `expr` is not true.
- `assertfalse(expr)` -- Fails if `expr` is not false.
- `asserteq(expected, actual)` -- Fails if values are not equal. Type-dispatched: the compiler selects the appropriate comparison (int, uint, float, string, bool, pointer) based on operand types.
- `asserteqf(expected, actual, epsilon)` -- Float equality within a tolerance. Fails if `|expected - actual| > epsilon`. All three operands must be `float32` or `float64`; a non-float operand is a compile error, not an implicit conversion.
- `assertnil(expr)` -- Fails if `expr` is not nil.
- `assertnotnil(expr)` -- Fails if `expr` is nil.
- `assertfail("message")` -- Unconditional failure with a message.

12. Expressions

```
Expression      = SimpleExpr [ RelOp SimpleExpr ] .
RelOp           = "=" | "<>" | "<" | ">" | "<=" | ">=" | "in" .

SimpleExpr      = [ "+" | "-" ] Term { AddOp Term } .
AddOp          = "+" | "-" | "or" | "xor" .

Term           = Factor { MulOp Factor } .
MulOp          = "*" | "/" | "div" | "mod" | "and" | "shl" | "shr" .

Factor         = "not" Factor | "-" Factor | "+" Factor
               | "address" "of" Factor | Primary .

Primary        = integer | float_literal | cstring | wstring
               | "true" | "false" | "nil"
               | SetLiteral | RecordLiteral
               | "(" Expression ")" | Designator | Intrinsic | TypeCast .

Designator     = ( ident | "varargs" ) { Selector } .
Selector       = "." ident | "[" Expression "]" | "^" | "(" [ ArgList ] ")" .

ArgList        = Expression { "," Expression } .

SetLiteral     = "[" [ SetElement { "," SetElement } ] "]" .
SetElement     = Expression [ ".." Expression ] .

RecordLiteral  = ident "(" FieldInit { "," FieldInit } ")" .
FieldInit      = ident ":" Expression .

TypeCast       = TypeExpr "(" Expression ")" .
```

Pointer Operations

- `address of expr` -- Returns a pointer to the operand.
- `expr^` -- Postfix (selector): dereference. Follows the pointer to its target.

⚡ 13. Intrinsic

```
Intrinsic      = LenExpr | SizeExpr | Utf8Expr | CStrExpr | WStrExpr
                | ParamCountExpr | ParamStrExpr | ExcCodeExpr | ExcMsgExpr .

LenExpr        = "len" "(" Expression ")" .
SizeExpr       = "size" "(" ( TypeExpr | Expression ) ")" .
Utf8Expr       = "utf8" "(" Expression ")" .
CStrExpr       = "cstr" "(" Expression ")" .
WStrExpr       = "wstr" "(" Expression ")" .
ParamCountExpr = "paramcount" "(" ")" .
ParamStrExpr   = "paramstr" "(" Expression ")" .
ExcCodeExpr    = "exccode" "(" ")" .
ExcMsgExpr     = "excmsg" "(" ")" .
```

text

📌 Note

`len` returns the length of strings, wide strings, and dynamic arrays.
`size` returns the byte size of a type or expression. `utf8` converts a wide string to a newly allocated, raw UTF-8 buffer (`char*`) - NOT a managed string; the buffer is owned by the caller. `cstr` returns a BORROWED raw UTF-8 `char*` pointing into an existing managed string's own storage - it allocates nothing and must never be freed, and the pointer is valid only while the owning string is alive. `wstr` is the UTF-16 counterpart of `cstr`: it returns a BORROWED `wchar*` that the runtime widens once and CACHES on the string itself, so repeat calls are free and the buffer must never be freed by the caller. Memory management (`new` / `dispose` / `getmem` / `freemem` / `resizemem` / `setlength`) is defined in Statements (Section 11).

🛒 14. Variadic Arguments

```
ParamList      = ParamDecl { ";" ParamDecl } [ ";" "..." ] | "..." .

VarArgsAccess  = "varargs" "." "next" "(" TypeExpr ")"
                | "varargs" "." "get" "(" Expression "," TypeExpr ")"
                | "varargs" "." "reset" "(" ")"
                | "varargs" "." "copy" "(" ")"
                | "varargs" "." "count" .
```

text

- `varargs.next(TypeExpr)` -- Retrieves and consumes the next variadic argument.
- `varargs.get(Expression, TypeExpr)` -- Retrieves the argument at the given index without advancing the cursor.
- `varargs.reset()` -- Resets the cursor back to the first argument.
- `varargs.count` -- Total number of variadic arguments passed.
- `varargs.copy()` -- Returns a new `varargs` object with a copied cursor position.

🔧 15. Unit Testing

Test blocks appear after the module's `end.` and are only compiled when the `@unittestmode on;` directive is active. When `@unittestmode on;` is active:

1. The compiler parses test blocks after `end.`
2. Each test block is compiled as a parameterless routine
3. The normal entry point is replaced with the built-in test runner

```
TestBlock      = "test" cstring [ "var" { VarDecl } ]
                "begin" StatementSeq "end" ";" .
```

text

Example

```
module exe mathlib;

@unittestmode on;

routine add(const a: int32; const b: int32): int32;
begin
  return a + b;
end;

routine mul(const a: int32; const b: int32): int32;
begin
  return a * b;
end;

initialize
  println("Module initialized");
end;

finalize
  println("Module finalized");
end;

end.

test "add returns correct sum"
var
  result: int32;
begin
  result := add(2, 3);
  asserteq(5, result);
end;

test "add handles negative numbers"
begin
  asserteq(-2, add(-5, 3));
  asserteq(-8, add(-5, -3));
end;

test "mul returns correct product"
begin
  asserteq(20, mul(4, 5));
  asserteq(0, mul(0, 100));
end;
```

text

16. C++ Interop

```
CppBlock      = "cppstart" CppTarget { character } "cppend" .
CppTarget     = "header" | "source" .

CppExpr       = "cpp" "(" cstring ")" .
```

text

Raw C++ Blocks

`cppstart` `header` ... `cppend` and `cppstart` `source` ... `cppend` inject raw C++ text directly into the generated header or source file. Everything between `cppstart` and `cppend` is captured verbatim -- no CPaskal parsing occurs inside the block. This is used for `#include` directives, inline C++ helpers, or any C++ code that cannot be expressed in CPaskal.

text

```
cppstart header
#include <iostream>
#include <vector>
cppend

cppstart source
void my_cpp_helper() {
    std::cout << "called from C++" << std::endl;
}
cppend
```

Raw blocks can appear at module level (among declarations) or inside statement blocks. The `header` target emits into the generated `.h` file; `source` emits into the generated `.cpp` file.

Inline C++ Expressions

`cpp(expr)` evaluates its string argument as a verbatim C++ expression in the generated code. It can appear anywhere a CPaskal expression is valid. The string is emitted as-is -- no escaping or validation is performed.

text

```
var x: int32 = cpp("100 + 23");
cpp("std::cout << 'C' << std::endl");
```

 17. Operator Precedence (Highest to Lowest)

Precedence	Operators
1 (highest)	<code>not</code> <code>-</code> (unary) <code>+</code> (unary) <code>address of</code>
2	<code>*</code> <code>/</code> <code>div</code> <code>mod</code> <code>and</code> <code>shl</code> <code>shr</code>
3	<code>+</code> <code>-</code> <code>or</code> <code>xor</code>
4 (lowest)	<code>=</code> <code><></code> <code><</code> <code>></code> <code><=</code> <code>>=</code> <code>in</code>

 Grammar Validation Checklist

Use this checklist when updating the grammar or adding syntax:

-  Lexical rules define the token shape before parser rules depend on it
-  Reserved words are listed before examples rely on them
-  New type forms appear in both the type grammar and the Language Reference
-  New routine syntax is reflected in declarations, statements, and examples where applicable
-  Operator changes update precedence and expression grammar together
-  Unit-test syntax matches the assertion helper documentation
-  Any new directive is added to the known directive table and the conditional compilation section

 Warning

 Keep grammar changes synchronized with examples. A grammar rule that accepts syntax not shown anywhere else is hard for users to discover, and an example that violates the grammar is worse than no example at all.

Runtime Library

CPaskal includes a small runtime library (`runtime.h` / `runtime.cpp`) that is automatically linked into every program. It provides the low-level machinery behind language features like exception handling, string conversion, set operations, memory management, CLI argument access, and unit testing. You never need to import or reference it directly -- the compiler generates calls to it as needed.

Exception Handling

CPaskal's `guard` / `except` / `finally` blocks compile down to runtime exception handling that catches both software exceptions (thrown with `throw` or `throwcode`) and hardware faults like division by zero and access violations.

Throwing Exceptions

```
throw("something went wrong");           // code defaults to 1 (RT_EXC_SOFTWARE)
throwcode(42, "custom error with code"); // user-defined error code
```

Catching Exceptions

```
guard
  // protected code that might fail
  throw("test error");
except
  println("caught: code={}, msg={}", exccode(), excmsg());
finally
  println("cleanup always runs");
end;
```

The `exccode()` intrinsic returns the integer error code. The `excmsg()` intrinsic returns the error message string. Both are valid only inside an `except` block.

Exception Codes

Code	Constant	Meaning
0	<code>RT_EXC_NONE</code>	No exception
1	<code>RT_EXC_SOFTWARE</code>	Software exception (default for <code>throw</code>)
2	<code>RT_EXC_DIV_BY_ZERO</code>	Division by zero
3	<code>RT_EXC_ACCESS_VIOLATION</code>	Null pointer or invalid memory access
4	<code>RT_EXC_STACK_OVERFLOW</code>	Stack overflow
5	<code>RT_EXC_INTEGER_OVERFLOW</code>	Integer overflow
6	<code>RT_EXC_ILLEGAL_INSTRUCTION</code>	Illegal CPU instruction
7	<code>RT_EXC_BUS_ERROR</code>	Bus error (alignment fault)
99	<code>RT_EXC_UNKNOWN</code>	Unknown or unrecognized exception

Note

Hardware exceptions (codes 2--7) are caught via Windows Vectored Exception Handling (VEH) on Windows and POSIX signals on Linux. The `guard` block handles both software and hardware exceptions uniformly.

String Conversion

The runtime provides conversion between UTF-8 and UTF-16 string encodings, which is essential for Windows API interop and wide-string handling.

utf8

Converts a wide string (`wstring`) to a UTF-8 encoded `char*` buffer. The caller owns the returned buffer:

```
var ws: wstring = w"Hello";
var s: pointer to char = utf8(ws);
// s is now a heap-allocated UTF-8 buffer -- caller must free
```

wstr

Converts a UTF-8 string to a wide character pointer. The runtime caches the result, so you do not need to free it:

```
var s: string = "Hello";
var ws: pointer to wchar = wstr(s);
// ws is a cached pointer -- no free needed
```

cstr

Borrows a raw `char*` pointer into managed string storage without allocation:

```
var s: string = "Hello";
var raw: pointer to char = cstr(s);
// raw points into s's internal buffer -- valid as long as s is alive
```

Tip

 Use `cstr` when passing a CPaskal `string` to a C function that expects `const char*`. Use `wstr` when a Windows API expects `const wchar_t*`. Use `utf8` when you need an owned copy of a wide string as UTF-8.

Memory Management

The runtime backs CPaskal's memory intrinsics with standard C allocation functions.

Typed Allocation: `new` / `dispose`

`new` allocates memory for a typed pointer and default-constructs the value. `dispose` frees the memory and sets the pointer to `nil`:

```
type Point = record x: int32; y: int32; end;

var p: pointer to Point;
new(p);           // allocate + zero-initialize
p^.x := 42;
p^.y := 99;
dispose(p);       // free + set to nil
```

Raw Allocation: `getmem` / `freemem` / `resizemem`

For unstructured byte buffers and C-style memory management:

```
var buf: pointer to uint8;
getmem(buf);           // allocate
resizemem(buf, 1024);  // resize to 1024 bytes
// ... use buf ...
freemem(buf);          // free
```

Intrinsic	Purpose
<code>new(ptr)</code>	Allocate and default-construct a typed pointer
<code>dispose(ptr)</code>	Free and set pointer to <code>nil</code>
<code>getmem(ptr)</code>	Allocate raw memory
<code>freemem(ptr)</code>	Free raw memory
<code>resizemem(ptr, size)</code>	Resize an existing raw allocation

 Note

`new` / `dispose` work with typed pointers and handle construction/destruction. `getmem` / `freemem` / `resizemem` work with raw bytes and do not initialize or finalize the memory. Use typed allocation when working with records; use raw allocation for byte buffers and interop.

 Command-Line Arguments

The runtime captures `main()` arguments at program startup and exposes them through two intrinsics.

```
println("program: {}", paramstr(0));    // program name / path
println("arg count: {}", paramcount()); // number of args (excludes program name)

var i: int32;
for i := 1 to paramcount() do
  println("arg {}: {}", i, paramstr(i));
end;
```

Intrinsic	Return Type	Purpose
<code>paramcount()</code>	<code>int32</code>	Number of CLI arguments (excludes the program name)
<code>paramstr(index)</code>	<code>string</code>	Get argument by index (0 = program name, 1..N = user args)

 Set Operations

Sets are implemented as efficient bitmask structures supporting up to 64 elements with an arbitrary base offset. The runtime provides the underlying operations that the compiler emits for set expressions.

```
var odds: set = [1, 3, 5, 7, 9];
var primes: set = [2, 3, 5, 7];

// Membership
if 5 in odds then
  println("5 is odd");
end;

// Union, intersection, difference
var both: set = odds * primes;      // [3, 5, 7]
var either: set = odds + primes;    // [1, 2, 3, 5, 7, 9]
var oddOnly: set = odds - primes;   // [1, 9]

// Equality
if odds <> primes then
  println("different sets");
end;
```

Operator	Operation
[elements]	Set literal (elements, ranges, or mixed)
in	Membership test
+	Union
*	Intersection
-	Difference
=	Equality
<>	Inequality



Tip

💡 Sets are stack-allocated bitmasks. Operations like union, intersection, and membership test compile down to single CPU bitwise instructions. Use them freely for flags, permission bits, and fast integer membership testing.

Variadic Functions

CPaskal supports C-style variadic functions using the `...` parameter syntax. The runtime provides a `varargs` structure for iterating over the variable arguments.

```
routine print_ints(const count: int32; ...);
var
  i: int32;
begin
  for i := 0 to varargs.count - 1 do
    println("{} ", varargs.next(int32));
  end;
end;
```

Inside a variadic function body, the implicit `varargs` object provides:

Member	Purpose
<code>varargs.count</code>	Number of variadic arguments passed
<code>varargs.next(Type)</code>	Extract the next argument as <code>Type</code> and advance

 Note

The caller must pass the correct count as the first parameter, and the types of variadic arguments must match what `varargs.next(Type)` expects. There is no runtime type checking on variadic arguments -- this is a low-level C ABI feature.

 Unit Testing

CPaskal has a built-in test framework. Test blocks are declared after `end.` and are only compiled when `@unittestmode on;` is active. The runtime registers each test and provides assertion functions.

Declaring Tests

```
module exe mylib;
@unittestmode on;

routine add(const a: int32; const b: int32): int32;
begin
  return a + b;
end;

end.

test "addition works"
begin
  asserteq(5, add(2, 3));
  asserteq(0, add(-5, 5));
end;

test "edge cases"
begin
  asserteq(0, add(0, 0));
  asserttrue(add(1, 1) > 0);
end;
```

Assertion Functions

Assertion	Purpose	Example
<code>assert(expr)</code>	Fail if <code>expr</code> is false	<code>assert(x > 0)</code>
<code>asserttrue(expr)</code>	Fail if <code>expr</code> is not true	<code>asserttrue(flag)</code>
<code>assertfalse(expr)</code>	Fail if <code>expr</code> is not false	<code>assertfalse(err)</code>
<code>asserteq(expected, actual)</code>	Fail if values are not equal	<code>asserteq(5, result)</code>
<code>asserteqf(expected, actual, epsilon)</code>	Float equality within tolerance	<code>asserteqf(3.14, pi, 0.01)</code>
<code>assertnil(expr)</code>	Fail if <code>expr</code> is not <code>nil</code>	<code>assertnil(p)</code>
<code>assertnotnil(expr)</code>	Fail if <code>expr</code> is <code>nil</code>	<code>assertnotnil(p)</code>
<code>assertfail("msg")</code>	Unconditional failure	<code>assertfail("not implemented")</code>

All assertions are non-aborting -- a failed assertion records the failure and continues. Multiple failures accumulate per test, and the test runner reports all of them at the end.

 Note

`asserteq` is type-dispatched. It works with integers, floats, strings, wide strings, booleans, and pointers. For floating-point comparison with tolerance, use `asserteqf` and pass an explicit epsilon value.

Console Initialization

The runtime initializes the console at program startup to ensure correct output behavior across platforms.

On Windows, this sets the console output code page to UTF-8 (`SetConsoleOutputCP(CP_UTF8)`) and enables ANSI escape sequence processing for colored output. On Linux, no special initialization is needed -- terminals handle UTF-8 and ANSI natively.

This happens automatically. You do not need to call any initialization function.

Output: print and println

CPaskal uses C++23 formatted output for console printing.

```
println("Hello, {}!", name);           // with newline
print("no newline here");              // without newline
println("{} + {} = {}", a, b, a + b);  // multiple values
```

The `{}` placeholder uses C++23 `std::format` syntax. Arguments are positional and type-safe -- the compiler matches each `{}` to the corresponding argument.

Intrinsic	Purpose
<code>print(fmt, args...)</code>	Formatted output, no trailing newline
<code>println(fmt, args...)</code>	Formatted output with trailing newline

Size and Length Intrinsics

Two intrinsics let you query the size and length of types and values at runtime.

```
println("int32 is {} bytes", size(int32));    // 4
println("Point is {} bytes", size(Point));    // 8 (two int32 fields)

var s: string = "hello";
println("length = {}", len(s));              // 5

var arr: array of int32;
setlength(arr, 10);
println("array length = {}", len(arr));      // 10
```

Intrinsic	Purpose
<code>size(type_or_expr)</code>	Byte size of a type or expression
<code>len(expr)</code>	Length of a string, wstring, or dynamic array
<code>setlength(arr, count)</code>	Resize a dynamic array to <code>count</code> elements

Intrinsics Summary

All intrinsics are built into the language -- they are not library functions and cannot be redefined or overridden. The compiler emits the appropriate C++23 code for each one.

Category	Intrinsics
Output	<code>print</code> , <code>println</code>
Strings	<code>utf8</code> , <code>wstr</code> , <code>cstr</code> , <code>len</code>
Memory	<code>new</code> , <code>dispose</code> , <code>getmem</code> , <code>freemem</code> , <code>resizemem</code> , <code>setlength</code>
Sizing	<code>size</code> , <code>len</code>

Category	Intrinsics
CLI	<code>paramcount</code> , <code>paramstr</code>
Exceptions	<code>throw</code> , <code>throwcode</code> , <code>exccode</code> , <code>excmsg</code>
Testing	<code>assert</code> , <code>asserttrue</code> , <code>assertfalse</code> , <code>asserteq</code> , <code>asserteqf</code> , <code>assertnil</code> , <code>assertnotnil</code> , <code>assertfail</code>
Variadics	<code>varargs.count</code> , <code>varargs.next</code>
Interop	<code>cpp</code>

 Tip

 Intrinsics look like function calls but are resolved at compile time. The compiler knows their exact semantics and emits optimized code for each one -- there is no function call overhead.

 Debugging

CPaskal has built-in support for source-level debugging. Debug builds produce native debug information (PDB on Windows, DWARF on Linux). On Windows, the compiler includes a native debugger that speaks the Debug Adapter Protocol (DAP), making it compatible with VS Code and other DAP-capable editors. On Linux, you can use the generated DWARF debug info with external debuggers such as GDB or LLDB.

 The `@breakpoint` Directive

Place `@breakpoint;` anywhere in your code to mark a debugger breakpoint location:

```
routine calculate(a: int32; b: int32): int32;
  var result: int32;
begin
  result := a + b;
  @breakpoint;    // debugger will stop here
  return result;
end;
```

The compiler does not emit any code for `@breakpoint`. Instead, it records the source location in the AST, and the debugger sets a native breakpoint at the corresponding machine address. This means breakpoints have zero overhead in release builds -- they are only active when the debugger is attached.

You can place `@breakpoint;` inside routines, init blocks, final blocks, and the main program body.

 Source-Level Debug Info

When you build with debug info enabled, the code generator emits `#line` directives in the generated C++ that map back to your original `.cpas` source files. This means the debugger shows your CPaskal source code, not the generated C++ -- you step through `.cpas` lines, set breakpoints on `.cpas` lines, and see `.cpas` filenames in stack traces.

The debug info format depends on the target platform:

Platform	Format	Description
Windows (<code>x86_64_windows</code>)	PDB	Microsoft Program Database, compatible with Visual Studio and VS Code
Linux (<code>x86_64_linux</code>)	DWARF	Standard Unix debug format, compatible with GDB, LLDB, and VS Code

Debug vs Release Builds

CPaskal supports four optimization levels that control whether debug info is included:

Level	Flag	Debug Info	Optimized
debug	-opt debug (default)	Yes	No
release_safe	-opt release_safe	No	Yes (with safety checks)
release_fast	-opt release_fast	No	Yes (maximum speed)
release_small	-opt release_small	No	Yes (minimum size)

The default is `debug`, so a plain `cpas myprogram -r` produces a debuggable binary. Use the conditional compilation symbols `DEBUG` and `RELEASE` to include or exclude code based on the build configuration:

```
@ifdef DEBUG
    println("debug build -- extra logging enabled");
@endif
```

 **Tip**

You do not need to pass `-opt debug` explicitly -- it is the default. Only specify an optimization level when you want a release build.

Launching the Debugger

Use the `-d` flag to build with debug info and immediately launch the integrated debugger:

```
cpas myprogram -d
```

This compiles the project, attaches the native DAP debugger, and runs the program under debugger control. The debugger will stop at any `@breakpoint;` locations in your source code.

 **Important**

The `-d` flag and `-r` (run) flag are mutually exclusive. Use `-d` when you want to debug, and `-r` when you just want to run.

 **Note**

The integrated debugger currently requires the `x86_64_windows` target (Windows only). On Linux, you can build with debug info (`-opt debug`) and use an external debugger such as GDB or LLDB.

DAP Protocol Support

CPaskal's native debugger implements the Debug Adapter Protocol (DAP), the same protocol used by VS Code, JetBrains, and many other editors for debugger integration. The debugger supports:

- **Breakpoints** -- set via `@breakpoint;` directives or interactively in the editor
- **Step In / Step Over / Step Out** -- standard stepping through your source code
- **Variable Inspection** -- view local variables, parameters, and global state
- **Call Stack** -- see the full call chain at any breakpoint or pause

Setting Up VS Code

To debug CPaskal programs in VS Code:

- 1. Build your program with the `-d` flag: `cpas myprogram -d`
- 2. The compiler launches the DAP server automatically
- 3. VS Code connects to the DAP server and presents the debugging UI
- 4. Breakpoints marked with `@breakpoint;` are hit as execution reaches them

 Complete Example

Here is a complete debuggable program that demonstrates breakpoints in both the main body and a routine:

```
module exe debug_example;

var globalCounter: int32 = 100;

routine add(a: int32; b: int32): int32;
  var sum: int32;
begin
  sum := a + b;
  @breakpoint;           // pause here to inspect sum, a, b
  return sum;
end;

begin
  var x: int32 = 10;
  var y: int32 = 20;
  @breakpoint;           // pause here to inspect x, y before the call
  var result: int32 = add(x, y);
  println("result = {}", result);
end.
```

Build and debug:

`cpas debug_example -d`

text

The debugger will stop first at the `@breakpoint` in the main body (before the call to `add`), then at the `@breakpoint` inside `add`, allowing you to inspect variables at each point.

 Code Style

CPaskal has simple, consistent conventions for naming, formatting, and file organization. Following these conventions keeps your code readable and consistent with the standard library and examples.

 Naming Conventions

Category	Style	Example
Types	PascalCase, no prefix	<code>Point</code> , <code>Color</code> , <code>HttpClient</code>
Variables	camelCase	<code>count</code> , <code>totalScore</code> , <code>isReady</code>
Constants	UPPER_CASE with underscores	<code>MAX_SIZE</code> , <code>DEFAULT_PORT</code> , <code>PI</code>
Routines	camelCase or snake_case	<code>add</code> , <code>getLength</code> , <code>make_point</code>
Module names	lowercase	<code>mathlib</code> , <code>netutils</code> , <code>sdl3</code>

 Note

 CPaskal does **not** use the `T` prefix on type names. If you are coming from Delphi or Object Pascal, write `Point` instead of `TPoint`, `Color` instead of `TColor`.

File Organization

Every CPaskal source file follows this structure:

```
module <kind> <name>;

// imports
import other_module;

// constants
const
  MY_CONST: int32 = 42;

// types
type
  MyRecord = record
    x: int32;
    y: int32;
  end;

// forward declarations (if needed)
forward routine helper(a: int32): int32;

// routines
routine helper(a: int32): int32;
begin
  return a * 2;
end;

// public API
public routine doWork(value: int32): int32;
begin
  return helper(value) + MY_CONST;
end;

// initialization (optional)
initialize
  println("module loaded");
end;

// finalization (optional)
finalize
  println("module unloaded");
end;

// main body (exe modules only)
begin
  println("Hello, CPaskal!");
end.
```

The module declaration is always first, followed by imports, then declarations (constants, types, variables, routines), optional init/final blocks, and the closing `end.` (with a main body for `exe` modules).

Comments

CPaskal supports two comment styles:

```
// Line comment -- everything after // to end of line

/* Block comment
   Can span multiple lines
   and can be /* nested */ safely */
```

Use line comments for short annotations. Use block comments for longer explanations or temporarily disabling code. The `( * *)` and `{ }` comment styles from traditional Pascal are **not** supported.

Formatting Guidelines

Indentation: Use consistent indentation (two or four spaces). Pick one and stick with it throughout your project.

Semicolons: Every statement ends with a semicolon. The `end` that closes a block also takes a semicolon (`end;`), except the final `end.` that closes the module.

Control structures: `if`, `while`, `for`, `match`, and `guard` always terminate with `end;`. There are no single-statement forms without `end`:

```
// Correct
if x > 0 then
  println("positive");
end;

// Also correct -- multiple statements
if x > 0 then
  println("positive");
  count += 1;
end;
```

Routine parameters: Separate parameters with semicolons. Group parameters of the same type when it reads naturally:

```
routine move(x: int32; y: int32; speed: float64): boolean;
```

Blank lines: Use blank lines to separate logical sections: between routines, between groups of related declarations, and before/after init/final blocks.

Visibility

Declarations are private by default. Use the `public` keyword to export a declaration from a module:

```
public const API_VERSION: int32 = 1;      // visible to importers
const INTERNAL_LIMIT: int32 = 256;        // private

public routine calculate(x: int32): int32; // exported
routine helper(x: int32): int32;          // private
```

When consuming imported symbols, always qualify them with the module name:

```
import mathlib;
var result: int32 = mathlib.add(2, 3);
```

✓ Example: Well-Structured Source File

```

module unit geometry;

// -- Constants --

public const ORIGIN_X: int32 = 0;
public const ORIGIN_Y: int32 = 0;

// -- Types --

public type
  Point = record
    x: int32;
    y: int32;
  end;

public type
  Rect = record
    left: int32;
    top: int32;
    width: int32;
    height: int32;
  end;

// -- Public API --

public routine makePoint(px: int32; py: int32): Point;
var result: Point;
begin
  result.x := px;
  result.y := py;
  return result;
end;

public routine area(const r: Rect): int32;
begin
  return r.width * r.height;
end;

public routine contains(const r: Rect; const p: Point): boolean;
begin
  return (p.x >= r.left) and (p.x < r.left + r.width)
    and (p.y >= r.top) and (p.y < r.top + r.height);
end;

// -- Module lifecycle --

initialize
  println("geometry loaded");
end;

end.

```

💡 Tip

💡 Keep routines short and focused. If a routine grows beyond a screenful, consider splitting it into smaller helpers. Group related routines together and separate groups with blank lines and a short comment header.

🔧 Common Tasks

Practical recipes for everyday CPaskal work. Each recipe is self-contained -- copy, adapt, and build.

Create a Hello World Program

The simplest CPaskal program:

```
module exe hello;
begin
  println("Hello, CPaskal!");
end.
```

Build and run:

```
cpas hello -r
```

text

Read Command Line Arguments

Access command line arguments with `paramcount()` and `paramstr()`:

```
module exe args_demo;
begin
  println("program: {}", paramstr(0));
  println("arg count: {}", paramcount());

  var i: int32;
  for i := 1 to paramcount() do
    println("  arg[{}] = {}", i, paramstr(i));
  end;
end.
```

💡 Tip

💡 `paramstr(0)` returns the program name. Arguments start at index 1. `paramcount()` excludes the program name.

Create and Use a Unit Module

Split code into reusable units. The unit:

```
// mathlib.cpas
module unit mathlib;

public routine add(a: int32; b: int32): int32;
begin
  return a + b;
end;

public routine multiply(a: int32; b: int32): int32;
begin
  return a * b;
end;

initialize
  println("mathlib loaded");
end;

end.
```

The consumer:

```
// main.cpas
module exe main;
import mathlib;
begin
  println("3 + 4 = {}", mathlib.add(3, 4));
  println("5 * 6 = {}", mathlib.multiply(5, 6));
end.
```

Important

 All imported symbols must be module-qualified: `mathlib.add(...)`, not just `add(...)`.

Create and Use a DLL

Build a shared library and call it from an executable.

The DLL:

```
// mylib.cpas
module dll mylib;

public routine clink double_it(const x: int32): int32;
begin
  return x * 2;
end;

end.
```

Build it:

cpas mylib

text

The consumer:

```
// app.cpas
module exe app;
routine clink double_it(const x: int32): int32; external "mylib";
begin
  println!("{}", double_it(21)); // prints 42
end.
```

Note

 The consumer declares the same signature with `external "mylib"` instead of a body. Use `clink` for C calling convention so the name is not mangled.

Call C Library Functions

Use the `external` clause with the `name` alias to call C functions:


```

module exe cffi_demo;

// Alias to avoid colliding with runtime headers
routine clink myabs(const n: int32): int32; external "c" name "abs";
routine clink mytoupper(const c: int32): int32; external "c" name "toupper";

begin
  println("abs(-42) = {}", myabs(-42));
  println("toupper(97) = {}", mytoupper(97));  // 'a' -> 'A'
end.

```

Important

🔑 Don't use stdlib function names directly as CPaskal routine names -- they collide with runtime headers. Always use the `name` clause to alias.

Handle Errors with `guard/except`

Use `guard` blocks for exception handling:

```

module exe error_demo;

routine risky_divide(a: int32; b: int32): int32;
begin
  if b = 0 then
    throw("division by zero");
  end;
  return a div b;
end;

begin
  guard
    var result: int32 = risky_divide(10, 0);
    println("result = {}", result);
  except
    println("error: code={}, msg={}, exccode(), excmsg()");
  finally
    println("cleanup complete");
  end;
end.

```

Use `throwcode` for custom error codes:

```
throwcode(42, "custom error with code 42");
```

Tip

💡 `guard` catches both software exceptions (`throw` / `throwcode`) and hardware exceptions (division by zero, access violations).

Work with Records and Pointers

Define a record, create instances on the stack and heap:

```

module exe records_demo;

type
  Point = record
    x: int32;
    y: int32;
  end;

routine print_point(const p: Point);
begin
  println("{}, {}".format(p.x, p.y));
end;

begin
  // Stack allocation with record literal
  var a: Point = Point(x: 10, y: 20);
  print_point(a);

  // Heap allocation
  var p: pointer to Point;
  new(p);
  p^.x := 30;
  p^.y := 40;
  print_point(p^);
  dispose(p);
end.

```

Use Sets

Sets support membership testing, union, intersection, and difference:

```

module exe sets_demo;
begin
  var evens: set = [0, 2, 4, 6, 8, 10];
  var primes: set = [2, 3, 5, 7];

  // Membership
  if 5 in primes then
    println("5 is prime");
  end;

  // Union
  var combined: set = evens + primes;

  // Intersection
  var even_primes: set = evens * primes;  // [2]

  // Difference
  var odd_primes: set = primes - evens;    // [3, 5, 7]

  // Ranges
  var digits: set = [0..9];
end.

```

Write and Run Tests

Add test blocks after `end.` with unit test mode enabled:

```

module exe testable;
@unittestmode on;

routine factorial(n: int32): int32;
var
  result: int32 = 1;
  i: int32;
begin
  for i := 1 to n do
    result *= i;
  end;
  return result;
end;

end.

test "factorial of 0 is 1"
begin
  asserteq(1, factorial(0));
end;

test "factorial of 5 is 120"
begin
  asserteq(120, factorial(5));
end;

test "factorial of negative returns 1"
begin
  asserteq(1, factorial(-1));
end;

```

Build and run tests:

```
cpas testable -r
```

text

Note

 When `@unittestmode on;` is active, the test runner replaces the normal entry point. All assertions are non-aborting -- failures accumulate and are reported at the end.

Build for Another Platform

Cross-compile for Linux from Windows (or vice versa) with the `-t` flag:

```
cpas myapp -t x86_64_linux
```

text

Use conditional compilation for platform-specific code:

```

module exe platform_demo;

#ifdef TARGET_WIN64
routine clink GetTick(): uint64; external "kernel32" name "GetTickCount64";
#endif

begin
#ifdef TARGET_WIN64
  println("Windows tick: {}", GetTick());
#elseif TARGET_LINUX64
  println("Running on Linux");
#endif
end.

```

 Tip

 Use `-r` with `-t` to build and run in one step: `cpas myapp -r -t x86_64_linux`. The Zig/Clang backend handles cross-compilation automatically.

Use Conditional Compilation

Guard code blocks with `@ifdef`, `@ifndef`, `@else`, `@elseif`, and `@endif`:

```
// Define your own symbols
#define VERBOSE

#ifdef VERBOSE
    println("debug: entering main loop");
#endif

// Check multiple conditions
#ifdef TARGET_WIN64
    // Windows-specific code
#elif TARGET_LINUX64
    // Linux-specific code
#else
    @message error "Unsupported platform";
#endif

// Undefine a symbol
#undef VERBOSE
```

Predefined symbols include `CPASKAL`, `WINDOWS`, `LINUX`, `DEBUG`, `RELEASE`, `BUILD_EXE`, `BUILD_DLL`, and `BUILD_LIB`.

Use Overloaded Routines

Overloaded routines require `cpplink` linkage:

```
module exe overload_demo;

routine cpplink describe(const x: int32);
begin
    println("integer: {}", x);
end;

routine cpplink describe(const x: float64);
begin
    println("float: {}", x);
end;

routine cpplink describe(const x: string);
begin
    println("string: {}", x);
end;

begin
    describe(42);
    describe(3.14);
    describe("hello");
end.
```

 Note

 `cpplink` enables C++ name mangling, which the linker needs to distinguish overloaded signatures. If you forget it, the compiler auto-promotes with a warning.

Use Dynamic Arrays

Allocate, resize, and iterate dynamic arrays:

```
module exe dynarray_demo;
begin
  var nums: array of int32;
  setlength(nums, 5);

  var i: int32;
  for i := 0 to len(nums) - 1 do
    nums[i] := i * i;
  end;

  for i := 0 to len(nums) - 1 do
    println("nums[{}] = {}", i, nums[i]);
  end;
end.
```

Embed Raw C++ Code

Use `cppstart` / `cpend` blocks for raw C++ and `cpp()` for inline expressions:

```
module exe cpp_demo;

cppstart header
#include <cmath>
cpend

cppstart source
double fast_sqrt(double x) { return std::sqrt(x); }
cpend

routine clink fast_sqrt(const x: float64): float64; external "c" name "fast_sqrt";

begin
  println("sqrt(2) = {}", fast_sqrt(2.0));
  var magic: int32 = cpp("42 + 58");
  println("magic = {}", magic);
end.
```

💡 Tip

💡 `cppstart header` injects into the generated `.h` file. `cppstart source` injects into the `.cpp` file. `cpp("expr")` emits the string verbatim as a C++ expression.

👉 Contributing

CPaskal is developed by tinyBigGAMES. Whether you are fixing a bug, improving documentation, improving examples, or proposing a feature, contributions are welcome.

Contribution	Best Way to Help
 Bug report	Open an issue with a minimal reproduction and the exact command used
 Feature idea	Describe the real use case first, then the proposed syntax or behavior
 Documentation fix	Point to the section and explain what was unclear or missing
 Test case	Include the smallest <code>.cpas</code> file that proves the behavior

Contribution	Best Way to Help
 Pull request	Keep the change focused and explain the before/after behavior

 Tip

-  Small, focused contributions are the easiest to review and the fastest to land.

 Support the Project

If CPaskal saves you time, helps you learn, or sparks something useful:

-  **Star the repo:** it costs nothing and helps others find the project
-  **Spread the word:** write a post, mention it in a community, or share a screenshot
-  **Join the community:** show what you are building and help shape what comes next
-  **Try examples:** real usage finds issues that synthetic tests miss
-  **Become a sponsor:** sponsorship directly funds development, examples, and documentation

 License

CPaskal is licensed under the **Apache License, Version 2.0**. See [LICENSE](#) for details.

Apache 2.0 is a permissive open source license that lets you use, modify, and distribute CPaskal freely in both open source and commercial projects. You are not required to release your own source code. Attribution is required: keep the copyright notice and license file in place.

 Links

-  [Homepage](#)
-  [GitHub](#)
-  [Discord](#)
-  [Bluesky](#)
-  [tinyBigGAMES](#)

 **CPaskal™** - Pascal elegance. C power. Batteries included.

Copyright © 2026-present tinyBigGAMES™ LLC
All Rights Reserved.